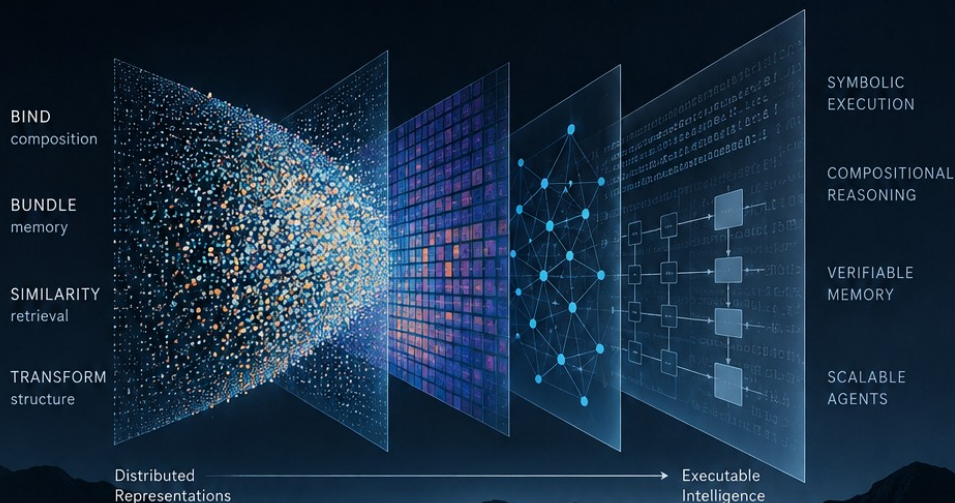


# Vector-Symbolic Intelligence in Practice

Experiments, Insights and Open Directions from an AI-Assisted Research Program

REPRESENTATIONS  
MEMORY  
REASONING  
ARCHITECTURES  
AGENTS  
TRUSTWORTHY AI  
WHAT COMES NEXT



HDC Architectures



Memory and Retrieval



Reasoning and Abduction



Towards VSA LLMs



Agentic Research

REAL EXPERIMENTS. REAL RESULTS. OPEN QUESTIONS.

ML CIRCUITS RESEARCH

Read online at  
<http://scriptahub.com/>

An experimental book.  
Your feedback is welcome.

# Vector-Symbolic Intelligence in Practice

---

*Experiments, Insights and Open Directions from an AI-Assisted Research Program*

**An AI analysis of fifteen human-directed, AI-assisted research repositories**

Research edition · 9 September 2026

This book is published openly at <http://scriptahub.com/>. It is an experimental research book: the underlying experiments were developed through human direction and AI coding agents, and this edition was produced by an AI auditing those AI-assisted experiments. Replications, counterexamples, corrections and research feedback are welcome.

The book is not an exhaustive survey of VSA/HDC. It reports selected mechanisms, reruns, diagnostic probes and open questions from the supplied repository snapshots. Claims are bounded by the evidence described in the Methods section and Appendices.

# Contents

|                                                                |     |
|----------------------------------------------------------------|-----|
| About this experimental research book                          | 4   |
| Methods and evidence boundaries                                | 7   |
| 1. Representation and operator architectures explored          | 9   |
| 2. From VSA algebra to language-model requirements             | 22  |
| 3. Operator semantics and algebraic mismatches                 | 30  |
| 4. Growing representations and explicit storage                | 38  |
| 5. Interference-aware memory placement                         | 47  |
| 6. Eos Gate: abductive property discovery and HDC decoding     | 55  |
| 7. Structural representations for events and text              | 64  |
| 8. Prediction, transformation, and executable compression      | 72  |
| 9. Exact reasoning and holographic retrieval                   | 80  |
| 10. Maintaining alternative hypotheses and selecting questions | 87  |
| 11. VSA-based routing of neural computation                    | 95  |
| 12. SOP Lang circuits as the current central architecture      | 102 |
| 13. Research conclusions and open experiments                  | 110 |
| A. Repository and execution evidence                           | 118 |
| B. Audit artifacts and reproduction boundaries                 | 123 |
| References and novelty boundaries                              | 126 |

# About this experimental research book

This book is itself part of the experiment it describes. The underlying research was directed by a human researcher and implemented over time with substantial assistance from coding agents and other AI systems. Fifteen repository snapshots were later supplied to another AI, whose task was not to continue the original narrative but to inspect the implementations, rerun feasible experiments, construct additional diagnostic tests where necessary, compare stated intentions with observed behaviour, and write a technically useful account of what appears to have been learned.

The book is made public at <http://scriptahub.com/> as an open research artifact. It is not intended to be a definitive monograph on Vector Symbolic Architectures or Hyperdimensional Computing. It is a selective record of a research programme that repeatedly asked whether VSA/HDC could provide useful building blocks for a language-capable, explainable and computationally efficient alternative or complement to contemporary neural language models. The purpose of publication is to expose the experiments, including ambiguous and negative results, to researchers who may recognise mechanisms worth testing further. Feedback, replications, counterexamples and alternative interpretations are explicitly welcome.

The central human assessment at the end of this research phase is cautious. The explored holographic representations and reasoning equations have not yet produced a sufficiently general and powerful substrate for unrestricted reasoning or language modelling. In particular, BIND and BUNDLE operations, even when

mathematically elegant, do not by themselves appear to cover the diversity of operations required for natural-language semantics, multi-step reasoning, revision, proof, planning and structured memory. This conclusion is not treated as final. Several failures may be caused by implementation choices, decoders, insufficient evaluation, or an incomplete operator set. It remains possible that useful extensions to VSA/HDC require representations or algebraic operations that have not yet been identified.

The repository collection is interesting precisely because it did not remain attached to one representation. It explored continuous real-valued vectors, dense binary hypervectors, sparse polynomial carriers, metric-valued vectors, elastic memories that grow, exact symbolic monomials, HRR memories based on complex unitary vectors, sharded memories, structural episodic representations, VSA-guided neural routing, and operator sets that begin to resemble executable programs. Some variants preserve fixed-size holographic storage but tolerate interference. Others reduce memory but alter algebraic properties. Others abandon fixed size to preserve more structure. Still others retain VSA only for retrieval or control while moving authoritative semantics into explicit structures.

The current architectural direction of the wider programme uses SOP Lang circuits as the central metaphor for executable knowledge and reasoning. The experiments in this book help explain that transition. They repeatedly show the value of making roles, premises, transformations, provenance, state and correction explicit. At the same time, they identify narrower roles in which VSA/HDC may remain valuable: associative addressing, approximate retrieval, interference-aware memory, similarity search, representation of bounded latent properties, and low-cost routing of computation.

This edition therefore has two goals. The first is empirical: state what the documentation appears to have intended, what the inspected code actually implements, what could be reproduced, what failed, and which measurements require reinterpretation. The second is generative: extract research questions that remain scientifically interesting after the claims are narrowed. A negative result is useful when it identifies the missing operator contract. A partial result is useful when it isolates a mechanism. A failed architecture may still contain a reusable memory policy, decoder, evaluation method, representation, or verification boundary.

The authorship model is deliberately unusual. The human researcher supplied the programme direction, research questions, feedback and many of the architectural intuitions. Coding agents implemented substantial portions of the repositories and, in some cases, left explicit reviews or design discussions. The AI writing this edition acts as an analyst of those human-AI experiments. Where the archives do not preserve reliable authorship metadata, the book does not invent it. Ideas are attributed only when evidence supports attribution. More generally, the book treats AI-assisted research as an engineering process that can be inspected, tested and corrected rather than as evidence of autonomous scientific agency.

# Methods and evidence boundaries

The analysis used the fifteen uploaded archive snapshots identified in Appendix A. They are treated as immutable source artifacts. The inspection covered the principal implementations, design documents, tests and saved results relevant to the questions discussed here. Selected repository test suites and experiments were rerun. Additional probes were created when a claim depended on a property that could be isolated more directly, for example whether a decoder rather than a representation caused an observed error, whether an algebraic inverse actually held, whether a metric was preserved by binding, or whether an evaluation metric could be satisfied vacuously.

Throughout the book, an observed result is one obtained by code inspection or execution during this audit. A reported result is one present in the supplied repository documentation or saved artifacts but not independently reproduced here. A proposed result is a research direction or architectural extension that remains untested. These categories are intentionally kept separate. Passing unit tests establishes consistency with the implemented test contract; it does not establish that the contract measures the intended cognitive capability.

The archive snapshots do not provide a complete live GitHub history, so the exact chronology of idea formation cannot always be reconstructed. No large-scale neural training campaign, paid external model evaluation, or exhaustive hardware benchmark was performed for this edition. Several experiments operate in synthetic worlds or bounded symbolic domains. Such experiments are scientifically useful because they permit precise controls and counterfactual checks, but they are not evidence of unrestricted natural-language competence.

The book is not exhaustive. It prioritises mechanisms that either produced an interpretable positive signal, exposed a useful failure mode, or suggest an unusual research direction. The main question is not whether VSA/HDC "works" in the abstract. The useful question is which representation, operator, memory policy or verification mechanism performs which task, under what assumptions, and at what cost.



# 1. Representation and operator architectures explored

The most important preliminary result of the repository collection is that "VSA/HDC" did not denote one architecture. The experiments repeatedly changed the carrier, the binding operation, the bundling operation, the similarity function, the memory policy and the role assigned to the vector representation. That variation is scientifically useful. It reveals which properties the researchers were trying to preserve and which costs they were willing to accept in exchange for those properties.

A useful way to read the experiments is to treat BIND and BUNDLE as contracts rather than as names. BIND is expected to compose two representations while retaining enough information to support some later operation, often an approximate or exact UNBIND. BUNDLE is expected to combine several representations into one memory-like object that remains related to its members. Different VSA families satisfy these expectations in different ways. The experiments in these repositories also test whether these two operations are sufficient. The evidence suggests that they are not: ordering, role identity, quantification, conditional structure, revision, provenance and program-like transformations repeatedly require additional mechanisms.

## 1.1 Continuous geometric vectors: addition, Hadamard binding and cosine readout

The spock repository represents concepts as dense real-valued vectors and gives geometric interpretations to a small set of primitive operations. Random concepts are created in a high-

dimensional continuous space. Add performs element-wise addition and is interpreted as superposition. Bind performs the Hadamard product. Negate multiplies by minus one. Distance converts cosine similarity into a score. Planning is framed as movement through the same geometric state space. The documentation further introduces a canonical truth direction and attempts to encode graded evidence through alignment or scaling. [R01]

The intended advantage is conceptual uniformity. The same numerical carrier can support similarity, superposition, continuous movement and compositional operations. Such an architecture is attractive if reasoning can be reduced to smooth transformations whose relevant distinctions are preserved by the geometry. It also maps naturally to efficient dense numerical computation.

The implementation, however, exposes an important contract issue. Hadamard binding is self-inverse only under specific operand families, such as bipolar vectors with coordinates restricted to plus or minus one. spock generates Gaussian-like real vectors, so multiplying a bound vector by one factor does not generally recover the other factor. The documentation sometimes uses the stronger bipolar intuition when describing the real-valued implementation. This does not make Hadamard multiplication useless; it means the claimed semantics of BIND must match the actual carrier.

A second audit result concerns confidence. If a vector is multiplied by a positive scalar and the next operation uses only cosine similarity, the scalar magnitude disappears. A state intended to represent stronger evidence can therefore become indistinguishable from a weaker state when interpreted through cosine alone. [A04] The broader lesson is that semantic meaning cannot be assigned independently to an operation and then

assumed to survive an arbitrary downstream readout. The representation, operators and decoders form one system.

This architecture remains interesting as an example of what a general geometric substrate would have to achieve. A future version could type quantities such as confidence separately from direction, use invertible or unitary binding families, or learn transformations under explicit algebraic constraints. What the current experiment does not support is the stronger idea that addition, Hadamard multiplication and cosine similarity already constitute a general reasoning algebra.

## 1.2 Dense binary HDC: XOR binding and majority bundling

HDC-RE and the reference backend of VSARepresentations use a more conventional binary HDC design. A concept is a dense high-dimensional bit vector. BIND is bitwise XOR. Because XOR is self-inverse,  $\text{bind}(\text{bind}(a,b),b)$  returns  $a$  exactly when all operands have the same geometry. BUNDLE uses a majority vote at each bit position. Similarity is derived from Hamming distance. [R02, R03]

The attraction of this representation is clear. Binding is computationally simple, exact and symmetric. A large binary vector is cheap to manipulate on a CPU using word-level bit operations. Random atom vectors are approximately orthogonal in high dimensions, and bundling can store a noisy superposition from which members may still be recognised by similarity. The model therefore offers a compact and mechanically clean separation between association and set-like superposition.

The sacrifice is also clear. Majority bundling is lossy. As more items are bundled, the contribution of any individual item competes with the aggregate of unrelated bits. Cleanup memory and a candidate vocabulary become part of the retrieval mechanism. Fixed-size

storage does not preserve arbitrary structured multiplicity or every role assignment without interference. XOR is also commutative, so ordering and directional relations require explicit role vectors, permutations, position encodings or another operation.

HDC-RE is particularly informative because its reasoning engine does not rely on the hypervector algebra alone. The inspected implementation combines HDC similarity with direct symbolic matches, rule chains and other explicit reasoning paths. [R02] This hybridisation is evidence that the project itself encountered a practical boundary: exact logical obligations are easier to control when represented explicitly, while HDC can still assist candidate retrieval and approximate matching.

Dense binary HDC therefore remains a useful baseline throughout the book. It supplies a strong, simple binding contract and a well-understood source of bundling noise. Newer representations can be evaluated by asking what they improve relative to this baseline and which properties they lose in exchange.

### **1.3 Sparse polynomial HDC: reducing storage by changing the carrier**

The sparse-polynomial backend in VSARepresentations attempts a much more aggressive memory reduction. Instead of thousands of dense coordinates, a vector is represented by a small set of 64-bit integer exponents. The documented default uses only a few exponents. BIND computes the Cartesian product of the two exponent sets and combines each pair with integer XOR. BUNDLE performs set union followed by sparsification when the number of exponents exceeds the configured budget. Similarity is Jaccard overlap. [R03]

The intended gain is substantial: a concept can be represented by a very small number of integers, and similarity can be computed over sparse sets. This architecture asks whether the useful algebraic behaviour of HDC can survive when most coordinates are removed and the representation is treated more like a sparse polynomial or set of symbolic exponents.

The critical result is that the algebra changes. The implementation comments suggest that XOR cancellation should support an inverse-like operation, but the full Cartesian set product does not generally satisfy the simple cancellation intuition once duplicate terms are collapsed by set semantics. A direct audit counterexample demonstrates failure even before the normal sparsification stage is required. [A04] Sparsification adds a second source of information loss because only a bounded subset of generated terms is retained.

This is a scientifically useful failure. It shows that a storage optimisation cannot be evaluated only by memory size or task accuracy. If the representation is intended to support algebraic reasoning, the operator laws themselves are part of the specification. A sparse carrier could still be valuable if its semantics are reformulated around approximate containment, probabilistic recovery or another well-defined operation. The mistake would be to retain the dense-XOR interpretation after replacing the underlying algebra with set-valued Cartesian products and pruning.

The experiment also suggests a broader research direction. Sparse representations may be better suited to explicit factorised structures than to imitating dense holographic vectors. If the sparse terms are treated as inspectable components and the system records how they were composed, the architecture begins to resemble an executable circuit or symbolic factor graph. That

direction trades some holographic elegance for auditability and controllable growth.

## 1.4 Metric-affine HDC: introducing numerical neighbourhoods

The metric-affine backend represents an atom as a fixed-length vector of bytes rather than bits. BIND remains byte-wise XOR, but BUNDLE becomes an arithmetic mean and similarity is based on normalised Manhattan distance. The motivation is to obtain a "fuzzier" geometry in which numerical closeness between coordinates is meaningful, rather than treating every bit mismatch equally. [R03]

This design is interesting because it explicitly searches for a richer notion of neighbourhood. A majority bit vector has a discrete geometry; a byte-valued space can represent graded coordinate differences, and averaging produces a natural centroid. If semantic similarity could be arranged to follow that geometry, the representation might support smoother interpolation while preserving the cheap XOR binding of binary HDC.

The audit shows why combining individually reasonable operators can create an inconsistent algebra. XOR is not an isometry of ordinary numerical Manhattan distance on byte values. Two vectors can be close in the byte metric, be bound by the same key, and no longer remain equally close. The implemented rounded arithmetic mean also does not satisfy a simple distributive relationship with XOR binding. [A04] Consequently, the similarity geometry and binding geometry are not aligned.

This does not imply that XOR and L1 distance can never coexist in a useful system. It means that the architecture must specify which properties are required after binding. If similarity is only used on

unbound atomic vectors, the incompatibility may be acceptable. If the system expects semantic neighbourhoods to remain stable through composition, a different group action, distance, or representation is required. The experiment therefore identifies a concrete open problem: design a carrier in which the desired binding operation is compatible with the similarity measure that gives meaning to the space.

## **1.5 Elastic metric-affine memory: growing storage instead of compressing indefinitely**

The elastic metric-affine backend retains the byte-valued atoms and XOR binding but changes BUNDLE substantially. Rather than reducing every superposition to one mean vector, it stores a sequence of bounded-capacity mean chunks. When more atoms are added, new chunks are allocated. Similarity between a bundle and an atom searches the chunk means and returns the best match. Binding a bundle with an atom transforms each chunk. Binding two bundles constructs a cross-product of their chunks. [R03]

The research objective is direct: relax the assumption that semantic memory must remain fixed in size. If the system learns more, it may allocate more state. This is closer to ordinary computer memory and to the human feedback motivating the broader programme. A useful memory architecture should be allowed to grow when compaction would destroy information required by future tasks.

The audit confirms that the implementation genuinely grows its typed-array payload in the measured configuration. It also confirms deterministic prefix compatibility for atom generation when a larger dimension is requested. [A04] These are positive engineering properties. They should not be confused with a proof that all earlier information can be reconstructed after arbitrary

expansion. Prefix-stable atom generation says that newly generated larger atoms are compatible with their shorter prefixes; it does not manufacture coordinates that were never stored in an old bundle.

The main sacrifice is that the representation is no longer a single fixed-size hologram. It keeps auxiliary state, and bundle-to-bundle binding can multiply the number of chunks. The architecture therefore shifts the research question from "How much can one vector remember?" to "When should memory allocate another chunk, preserve source state, or compact?" That is a more general systems question and is likely to remain relevant even outside VSA/HDC.

An especially promising continuation would make allocation obligation-driven. The system could grow when retrieval margins fall below a threshold, when a new semantic role cannot be represented without collision, or when provenance must be preserved for verification. Such a policy is not demonstrated by the current repository, but the elastic representation provides an experimental substrate for testing it.

## **1.6 Exact monomial representations: eliminating holographic noise at the price of expansion**

The exact backend in VSARepresentations makes the most radical change. A representation is a list of exact monomials, implemented as bit sets encoded in BigInt values. BUNDLE forms the set union of terms. BIND forms all pairwise unions of atom bits. UNBIND is implemented as a quotient-like residual operation: if the queried component is contained in a composite term, the query bits are removed and the residual is returned. [R03]

This design removes the usual collision noise of a fixed-width holographic superposition. A query can recover explicit residual



structures rather than infer them only from vector similarity. The architecture is therefore useful for asking whether the difficulty attributed to VSA is fundamentally caused by approximate representation or by the combinatorics of structure itself.

The result is instructive. Exactness does not provide free capacity. Binding two bundles expands their combinations. A simple audit construction with twelve independent binary alternatives yields 4,096 explicit terms. [A04] Repeated factors are also not preserved in the same way as a group inverse because set union is idempotent. These are not implementation defects; they follow from the chosen semantics.

The representation exposes a fundamental trade-off. A fixed-size holographic vector can compress many structures but pays in interference and ambiguous cleanup. An exact enumerative structure eliminates that ambiguity but may expand exponentially. A third option is to preserve a factorised expression and postpone expansion until a query requires it. Once again, that third option resembles a circuit: retain the composition graph, execute only the needed branch, and preserve provenance of the transformation.

This experiment is one of the strongest conceptual bridges from VSA research to SOP Lang. It suggests that the object worth preserving may not be a final vector at all, but an executable expression whose nodes can use vector operations when vector operations are advantageous.

## **1.7 HRR with complex unitary vectors: approximate unbinding plus interference-aware sharding**

VSANash returns to a classical Holographic Reduced Representation design. Symbols and memories are represented in the frequency domain with complex vectors. Binding corresponds

to element-wise complex multiplication, equivalent to circular convolution in the time domain. Keys are constructed to be unitary so that inverse binding is stable. Facts are superposed by addition. Retrieval unbinds a subject-relation key from the memory and uses cosine similarity against candidate entity vectors. [R04]

The representation is mathematically better matched to its intended inverse than the early real-valued Hadamard implementation. It also makes interference explicit: every unrelated fact superposed into the same memory contributes residual energy after unbinding. VSANash does not attempt to remove that property algebraically. Instead, it asks whether a set of bounded HRR memories can be organised so that mutually interfering facts are placed in different shards.

That change of focus is significant. The representation remains holographic, but the main algorithm becomes a memory-allocation policy. The repository compares simple hashing, packing, collision-aware heuristics, equilibrium-style placement and multi-write variants. In a rerun of the low-load scenario, the retrieval-aware placement produced a substantial improvement in answer recovery compared with simple key hashing. The same approach did not eliminate collapse under heavier load or more difficult multi-hop conditions. [A03]

VSANash also contains bipolar and XOR-majority backends, providing discrete controls against the HRR implementation. These controls are important because a memory-placement gain should not be attributed automatically to circular convolution or complex numbers. The potentially interesting contribution is the interference-aware placement strategy, which could be applicable to several associative-memory families.

This experiment therefore changes the research question from "Which binding operator is best?" to "How should an approximate

memory be allocated, routed and replicated so that its error remains measurable and controllable?" For agentic systems with long-lived memory, that may be more practically important than finding another mathematically elegant BIND.

## **1.8 Beyond BIND and BUNDLE: explicit transforms, structural indexes and routing**

Several repositories gradually reduce the semantic responsibility assigned to the VSA carrier. This is an important architectural result in its own right.

BSP explores explicit bitset transformations including XOR, AND, OR, permutation, identity and negation, together with learned or discovered transform patterns. The documentation treats these operations as an instruction set for geometric compression rather than insisting that all semantic transformation be expressed through one BIND/BUNDLE pair. [R09] The experiment is incomplete as a general language model, and one evaluation metric was found to be defective, but the operator-expansion idea remains relevant. Natural-language reasoning may require a library of typed transformations whose applicability is learned or planned.

VSABrains2 and VSAText retain more explicit event and role structure. In these systems, exact fields, role-value maps, schemas or episodic records coexist with vector-like addressing or retrieval mechanisms. [R05, R06] This sacrifices the ambition that a single holographic object is the canonical representation of meaning. In return, the architecture can preserve distinctions such as subject, object, event order and provenance more directly. The audit also shows why that matters: an encoder that collapses role order cannot recover it later, regardless of the quality of the vector search.

VSA-PathMoE narrows the role of VSA even further. It uses a vector-symbolic representation to select which neural experts should process a request. The VSA state is a control signal rather than the repository of all semantic content. [R15] This may be a productive endpoint for some of the research: VSA can contribute deterministic, compositional or low-cost routing without being required to implement every form of reasoning.

These architectures point to a general design principle. VSA/HDC should be evaluated as one computational resource among several. A system can use holographic vectors for association, exact structures for semantic identity, circuits for transformation, formal solvers for proof and neural models for perception or generation. The scientific question becomes how these components exchange typed information and how each result carries enough provenance to be checked.

## 1.9 What the representation search has established so far

No representation in the supplied experiments simultaneously demonstrates fixed-size storage, exact structural recovery, general algebraic reasoning, efficient learning, open-domain language competence and simple CPU execution. This is not surprising; those requirements pull in different directions. The value of the experiments lies in making the trade-offs concrete.

The continuous geometric approach offers smooth similarity but weak operator guarantees. Dense binary HDC offers exact XOR inversion but noisy majority storage. Sparse polynomial HDC reduces storage but changes the algebra and depends on pruning. Metric-affine HDC enriches neighbourhood structure but combines a metric with a binding operation that does not preserve that metric. Elastic memory preserves more information by growing. Exact monomials remove holographic noise but expose

combinatorial expansion. HRR provides a principled approximate inverse yet remains constrained by superposition interference. Sharding improves part of that problem without solving high-load reasoning. Structural hybrids and VSA routing obtain usefulness by assigning vectors a narrower role.

This pattern supports the human research conclusion that the programme should not remain committed to VSA/HDC as the unique central substrate. It does not support abandoning VSA/HDC entirely. Several experiments suggest narrower, testable contributions, and the possibility of new operators remains open. In particular, an unexplored representation could be important if it satisfies a set of operator contracts that the current backends cannot satisfy together: role-sensitive composition, bounded or factorised growth, similarity compatible with binding, reversible update, and explicit provenance.

The next chapters examine the experiments by scientific function rather than by representation family. The representation catalogue in this chapter should remain in view: many apparent reasoning successes or failures can only be interpreted correctly after identifying the exact carrier, BIND, BUNDLE, similarity and cleanup semantics used in that experiment.

## 2. From VSA algebra to language-model requirements

This chapter examines the original systems-level question behind the repository collection: whether a sufficiently small family of vector operations could serve as the computational core of a language-capable system. The experiments do not implement a single architecture. They test different obligations—identity, retrieval, composition, prediction, proof, adaptation, and control. The main result is therefore architectural: a useful language system requires several distinguishable kinds of computation, and success on one does not establish a general algebra for the others.

### 2.1 A language model requires several computational obligations

Consider a simple instruction: summarise what happened to an object, explain why it moved, and distinguish what the narrator knows from what a character believes. Even this small task contains several different computations. The system must identify the object across sentences, preserve the order of events, distinguish an event from a report about it, decide which causal relation is warranted, and select information appropriate for the requested summary. Producing the final sentence is another computation. A representation may make one of these operations cheap while making another difficult or impossible to recover.

The fifteen repositories approach different parts of that problem. The spock and HDC-RE experiments investigate whether a small algebra over vectors can support reasoning. VSARepresentations asks whether the carrier itself should change: binary vectors,

sparse expressions, metric vectors, elastic collections, or exact symbolic terms. VSANash studies where to place distributed memories so they interfere less. VSABrains2 and VSAText organise episodic and narrative information. Eos Gate, in *vsaAbduction*, tests whether a property can be inferred from observations and used to predict or repair configurations. BSP, SomaVM, and VSAVM explore prediction, compression, local adaptation, or executable units. CNL, UBHNL, and HoloProof move toward explicit semantic representations and controlled execution. *autoResearchLib* maintains alternative explanations and chooses questions. *vsaLLM* uses VSA to select neural computation rather than replace it. [R01–R15]

Calling all of these attempts “a VSA language model” is understandable as a research direction but misleading as an experimental description. Their outputs, failure conditions, and success criteria differ. A fact-retrieval system answers from a finite vocabulary. A theorem prover checks whether an assertion follows under a specified logic. A byte predictor estimates continuations. An abductive learner identifies a member of a restricted hypothesis class. Each can contribute to a language-capable architecture. None acquires the other capabilities merely because its state can be called a hypervector.

This distinction changes the meaning of generality. An operator might be general over a mathematical domain: XOR applies to every pair of equal-length binary strings. That does not make it a general operation over meanings. To obtain a reasoning step from XOR, the encoding must map the relevant semantic relationship to something that XOR preserves. A second operation may require a different encoding or additional structure. The programme repeatedly encounters this gap between an operation defined everywhere and an operation useful everywhere.

The surveyed VSA literature already describes multiple models, transformations, and cognitive applications rather than one universal algebra. It also makes clear why the family is attractive: distributed representations can combine symbolic structure with similarity-sensitive computation. Those are useful starting properties, not a specification for unrestricted language understanding. The appropriate comparison is therefore not between “symbolic” and “vector” intelligence as competing essences, but between concrete mechanisms with stated obligations. [L01, L02]

An instructive obligation is role preservation. If “Alice helps Bob” and “Bob helps Alice” are encoded identically, no subsequent decoder receiving only that encoding can reliably distinguish them. This is not a shortage of training data in the decoder. It is an equality introduced by the encoder. Another obligation is calibration. If confidence is multiplied into a vector and the next stage uses only cosine similarity, the confidence may disappear. A third is coverage. If a solver proves a statement about a retrieved subset, its answer may not apply to the entire knowledge base. These failures occupy different layers and require different repairs.

The user's dissatisfaction with a universal holographic reasoning equation becomes more precise when expressed in these terms. A useful language architecture must preserve the distinctions demanded by its tasks, provide operators that act correctly on those distinctions, and control how partial results become accepted answers. The supplied code contains evidence for failures at all three layers. It also contains useful ways to address them, often by introducing explicit structures that no longer resemble a single fixed-size holographic state.

This does not make VSA irrelevant. A language system spends substantial effort searching, associating, selecting, and reusing.



Approximate distributed representations may be well suited to those operations. The question is whether they should also be the sole authority for identity, scope, proof, and memory updates. The experiments increasingly suggest that these responsibilities should be separable. That is an architectural conclusion supported by the distribution of observed strengths and weaknesses, not a theorem excluding other architectures.

## 2.2 What code inspection and reruns changed

The supplied snapshots made a different kind of book possible. Instead of repeating repository descriptions, I could ask whether the code implements the claimed operation, whether the test measures the claimed ability, and whether a failure survives a controlled change. The investigation covered the inventory of all fifteen archives, selected core implementation paths and design documents, test runners, saved experiment artifacts, and feasible reruns. It was not an exhaustive security audit or a line-by-line review of every file. [A01, A02]

The first lesson was that green test suites do not have a common meaning. spock's runner completed 350 checks. VSAText completed 268. HoloProof completed 73 tests, yet its separate smoke evaluation produced twelve solver errors because the expected external solver executables were unavailable. UBHNL's default development command exited successfully while explicitly reporting that no development tests had been implemented; its separate typed-AST tests did pass. These are different observations. Adding all of them into one impressive number would remove precisely the distinctions the audit needs to preserve. [A02, A03]

The second lesson was that failure counts can be equally misleading. VSARepresentations passed 807 of 808 tests in the selected suite; the failing test concerned a missing CommonJS entry

point. `autoResearchLib` passed eighteen of nineteen tests; the remaining test expected a particular sibling project layout absent from these archives. `VSAVM` passed 305 of 307 tests, and the two failures were caused by a local JavaScript declaration-order error in generation code. None of these observations establishes a mathematical limit of VSA. They establish packaging, environment, or implementation problems. [A02]

For `VSAVM`, a small causal repair was possible. The code used `budgetMs` before its `const` declaration, producing a temporal-dead-zone exception. Moving that declaration above the use, without changing the learning algorithm, made the complete selected suite pass all 307 tests. The patch and both logs are included in the companion material. This is stronger evidence than merely guessing that “engineering may be the problem.” In this case a specific engineering error caused the observed failures. It is also narrower than claiming that the repair created a competent language model. The passing tests still establish only what those tests exercise. [A07]

Other results moved in the opposite direction. `BSP`’s abstract-negation evaluation reported perfect accuracy, but the scoring rule counted a case as correct when the forbidden token was absent from the candidate list. A new probe supplied an untrained engine that returned no candidates. It received 1,000 correct judgments out of 1,000 cases. The headline result survived while its intended interpretation did not. This was not a minor rounding problem. The measurement failed to require any positive competence. [A06]

The `Eos Gate` experiment produced a more encouraging reversal. The original implementation learned a useful symbolic invariant, but its `HDC` readout lost accuracy. Inspecting the encoding showed that it remained a full-rank linear transformation of twelve features in the tested configurations. Replacing an approximate

dot-product decoder with a dual readout recovered the target quantity to numerical precision. Across twenty-five sampled codebooks, the new readout classified every tested split correctly. Here the weaker result had concealed recoverable information. [A05]

These observations set the tone of the book. It is not enough to classify repositories as successes or failures. The important object is the causal account: an encoder erased a distinction, a decoder introduced cross-talk, a scoring rule rewarded silence, a runtime exception interrupted generation, a symbolic validator rescued a candidate, or a memory-placement policy reduced interference. Once the cause is identified, the next experiment becomes much clearer.

That is also the principal value of using an AI analyst after AI coding agents. The additional AI should not merely produce another confident explanation of the same artifacts. It should create interventions that distinguish competing explanations. This edition contains some such interventions, but the same standard applies to them. Their source, inputs, outputs, and limits are exposed so that another researcher can challenge the interpretation rather than accept the authority of the prose.

## **2.3 How novelty is assessed in this book**

A research programme can generate valuable novelty without producing a new universal architecture. One kind of novelty is a mechanism: a different way to allocate memory, decode a representation, or combine operators. Another is an empirical fact about a specific implementation. A third is an explanation that replaces an earlier account of why something worked or failed. A fourth is a research question made precise enough that an experiment can settle it. The book keeps these categories separate.

The new dual-readout experiment is primarily an explanatory result. Its mathematics is ordinary linear algebra, and related VSA work already studies linear readout rather than similarity search. Its value here is to show that the observed Eos error is not evidence of unavoidable information loss in the tested encoding. The underlying operation should not be advertised as newly invented. The causal diagnosis is new to this audit and changes how that experiment should be interpreted. [A05; L06]

The negation probe is a measurement result. It demonstrates a property of an actual scorer: an empty answer set satisfies its criterion. No new theory of negation is claimed. Yet the result is important because it changes the evidence available for selecting a research direction. A programme that mistook this score for reasoning ability might reinforce the wrong mechanism. A programme that fixes the scorer learns what its system can actually do.

VSANash is closer to a candidate mechanism contribution. Its placement policies use retrieval behaviour and load when deciding where facts should live. In the rerun low-load configuration, an equilibrium-style policy improved top-ranked-set retrieval over simpler sharded controls. This is not enough to establish publication-level originality or general superiority. It does identify a concrete coupling worth testing: storage placement informed by the very interference that later makes retrieval difficult. [R04; A03]

The elastic and exact representations raise another kind of question. They do not show that unlimited knowledge fits into a fixed hypervector. Instead they expose different ways for representation state to grow. New chunks preserve earlier contributions. New symbolic atoms extend an exact universe. Cartesian combinations preserve alternatives while risking exponential expansion. The useful contribution may lie in a policy

that chooses among those growth modes according to the operations a task requires. That policy is proposed later in the book; it is not silently credited to code that only implements individual backends.

Finally, the move toward SOP Lang circuits is an architectural hypothesis informed by the collection. It is not a claim to have invented dataflow graphs, theorem-prover interfaces, provenance, or program synthesis. Those ideas have substantial histories. The proposed contribution is a particular division of responsibility: executable semantic structure as the canonical object, approximate representations as optional accelerators, and explicit obligations at every conversion between them. The novelty question concerns whether that composition produces better behaviour, not whether the word “circuit” is new.

The phrase “new to this audit” is deliberately modest but not dismissive. A false invariant discovered in source code is a real finding. A decoder that removes an apparent representation limit is a real finding. A question policy that becomes more confident while eliminating the correct hypothesis is a real finding. Their scientific value increases when their scope is exact and their reproduction is inexpensive.

The human researcher supplied the programme's ambition and the decision problem that now matters: whether to continue centring the architecture on VSA/HDC. Earlier agents helped construct implementations and documents; particular reviews acknowledge AI authorship. This AI contributes an inspection, several new probes, and a synthesis. Without complete development records, it would be inappropriate to assign every mechanism to a specific mind. What can be assigned reliably is responsibility for claims in this edition. The remainder of the book is an attempt to make those claims worth examining.

### 3. Operator semantics and algebraic mismatches

The experiments in `spock`, `HDC-RE`, and `VSAR` representations test whether the semantics assigned to vector operations are actually preserved by the implemented carrier, similarity measure, and decoder. The intended goal was to make reasoning geometric or holographic while retaining simple composition. Direct probes show several mismatches: information represented in vector magnitude can disappear under cosine comparison; sparse set-XOR does not automatically provide the illustrated inverse; and XOR binding is not an isometry for the byte-valued metric used by the metric-affine backend. These are useful negative results because they identify operator contracts that any successor architecture must state explicitly.

#### 3.1 Magnitude information versus cosine readout

The `spock` repository is a useful place to begin because its numerical kernel is small. It exposes operations corresponding to addition, binding, negation, a distance-like similarity, movement, modulation, identity, and normalisation. The implementation is inspectable enough to compare mathematical behaviour with the semantic language used in the accompanying theory documents. The attraction is clear: a limited set of fast operations might support a larger family of cognitive transformations. [R01]

The first difficulty appears when confidence is represented by positive scaling. Suppose a vector describes a proposition and a factor of 0.7 is intended to express reduced confidence. The next stage compares this vector with another using cosine similarity. Cosine divides the dot product by the product of the vector norms.

Multiplying one vector by a positive scalar multiplies both its contribution to the numerator and its norm in the denominator by that scalar. The factor cancels.

Written as an executable contract, the relevant identity is  $\text{cosine}(c * x, y) = \text{cosine}(x, y)$  for nonzero vectors and positive  $c$ . The audit tested the actual primitive with a simple vector and scale factors 0.1, 0.3, 0.7, and 1.0. Its cosine with the original remained one to numerical precision. The transformed similarity used by the implementation also remained one. A downstream decision that sees only this similarity cannot tell these confidence values apart. [A04]

This is not an argument against using vector magnitude. Magnitude can carry information when later operations read it. It is an argument against claiming that a particular pipeline preserves a confidence variable when that pipeline deliberately removes the relevant degree of freedom. A repair might keep confidence in a separate field, use a magnitude-sensitive score with explicit calibration, or encode uncertainty through a distribution rather than positive scaling. Each choice changes the semantics and should be tested as such.

The example exposes a broader risk in AI-generated theory documents. Operations receive meaningful names, then the names begin to act as evidence. “Modulate” sounds like changing belief strength. “Distance” sounds like measuring semantic disagreement. “Normalise” sounds like a harmless numerical adjustment. But if the representation of uncertainty is magnitude, normalisation is not semantically harmless. The question is not whether the functions have plausible interpretations independently. It is whether the composition preserves the intended variable.

Similar care is needed with negation. Replacing a vector with its negative creates an opposite direction under cosine. That is a well-defined geometric operation. Logical negation, however, changes

the truth condition of a proposition while preserving its content as the subject of denial. The negative of a vector for “Alice is in the kitchen” does not automatically identify the appropriate alternatives, distinguish explicit denial from missing evidence, or implement the scope of “Alice does not believe that Bob left.” A geometric opposite becomes logical negation only within a verified encoding and reasoning scheme.

The same limitation applies to addition. Adding vectors may bundle evidence or mix features, but calling the result Bayesian does not establish a Bayesian update. Such an update requires a specified probabilistic model, likelihoods, and a rule for combining dependent or independent evidence. The inspected primitive operations do not supply those ingredients merely by addition. The responsible conclusion is not that probabilistic interpretation is impossible; it is that the relevant model remains an additional obligation.

What survives from spock is useful. A small kernel, explicit operation calls, and a traceable execution structure are good engineering foundations. The audit's 350 successful checks support the implemented behaviours exercised by its test suite. They do not validate every semantic equation in the theory documents. The interesting next step is to convert each intended interpretation into a contract that composes: which state variable encodes confidence, which operations preserve it, and which decision rule actually reads it? That is already closer to a typed executable circuit than to an unrestricted analogy between geometry and thought. [A02]

### 3.2 Hybrid symbolic support inside HDC reasoning

HDC-RE explores reasoning more explicitly. Its proof path combines vector similarity with direct matches, graph-like transitive chains, backward application of rules, and explicit treatment of certain



incompatible categories. This makes the system more capable than a bare similarity search. It also makes attribution more difficult. When the final answer is correct, which component established it? [R02]

The inspected proof implementation uses thresholded direct matches and then additional structured procedures. Relations are not all treated as arbitrary semantic objects governed by one universal equation. Some paths have specific transitive behaviour; rules carry symbolic conditions. In such a system, hypervectors may propose candidates or help find related assertions, while the decisive inference occurs in the structured rule machinery. This is not a defect. It is a hybrid architecture. The defect would be to describe its success as evidence that the vector operation alone performed the deduction.

A familiar example makes the distinction concrete. If the knowledge base states that every amber object is warm and that a particular object is amber, a symbolic rule application can derive that it is warm. An approximate representation might retrieve the relevant rule or object. The proof nevertheless depends on variable binding, the implication's direction, and the match between its premise and the known fact. A large cosine between two terms does not supply those obligations. A trace should say whether a candidate was merely similar or whether a substitution satisfied the rule.

The implementation's thresholds deserve similar scrutiny. A score above a chosen level may be useful for candidate admission, but it is not automatically a calibrated probability of truth. Scores depend on the representation family, the number of bundled items, the composition history, the cleanup vocabulary, and the type of operands being compared. The later metric-backend probes show this dependency explicitly. A threshold can remain useful in a

bounded deployment, but its meaning must be measured under that deployment's conditions.

The HDC-RE test run passed 476 tests and failed thirteen. Several failures concern expectations about APIs, naming, lexing, or rule-tracking behaviour rather than a clean test of the entire reasoning hypothesis. They should be repaired and classified before being used as scientific evidence. Equally, the large number of passing tests cannot be treated as a sample of unrestricted reasoning tasks. It is evidence that much of the implemented machinery behaves according to its existing examples. [A02]

A more informative experiment would keep the semantic parser, facts, rules, and acceptance checker fixed while replacing candidate generation. One version would use the current HDC route. Another would use exact inverted indexes over predicates and argument positions. A third would use term indexing for structured rule matching. A fourth could use a learned or lexical retrieval baseline if text remains part of the task. The comparison would measure missing candidates, spurious candidates, downstream proof work, latency, and total memory. Final answer accuracy alone would hide whether one retrieval method merely caused more work for the same exact checker.

This distinction between candidate generation and authority becomes one of the programme's strongest reusable insights. Approximate retrieval does not need to be a theorem prover to be valuable. It needs to reduce the cost of reaching a theorem prover's relevant inputs without losing indispensable evidence too often. Conversely, an exact checker cannot compensate for evidence that never reaches it unless it has a coverage or fallback mechanism. These are complementary responsibilities, not interchangeable brands of intelligence.

The novelty opportunity is therefore more specific than “reasoning in hypervectors.” It concerns a controlled interface between approximate representations and executable rules. The interface should expose what was retrieved, why it was admitted, what exact operation accepted it, and what remains unsearched. In a future SOP architecture, these would be different circuit nodes and different result types. The present repositories do not implement that unified contract, but they show why it would be useful.

### 3.3 When binding and similarity encode incompatible geometries

VSARepresentations makes the algebraic problem particularly visible because several backends implement superficially similar operations. A generic interface encourages substitution: encode, bind, bundle, compare, and unbind. Yet identical method names do not imply identical laws. An execution engine can become incorrect when it assumes that every backend preserves properties true of only one representation family. [R03]

The metric-affine backend provides an exact small counterexample. Its coordinates are bytes. Binding uses bitwise XOR, while similarity is based on numerical distance between byte values. XOR preserves bitwise relationships, but it does not preserve ordinary distance on the ordered numbers from zero to 255. Take coordinates 127 and 128. They differ by one, so the normalised distance-based similarity is about 0.996. XOR both with 127 and they become zero and 255, whose similarity is zero. The audit reproduced this with the actual implementation. [A04]

No amount of dimensionality fixes this one-coordinate incompatibility. Repeating the coordinate would repeat the counterexample. Using more random coordinates may hide the effect on an average workload, but it does not establish the missing

algebraic law. If an inference step depends on binding preserving metric closeness, the backend does not satisfy that contract. The options are to change the metric, change binding, restrict the operand distribution and validate a probabilistic property, or stop depending on the law.

Bundling introduces another mismatch. The backend combines coordinates through a rounded mean. XOR does not generally distribute over that operation. For values zero and two, their mean is one. XOR the mean with one and the result is zero. XOR the original values with one first, obtaining one and three, and their mean is two. Thus `bind(bundle(x, y), key)` and `bundle(bind(x, key), bind(y, key))` disagree in the actual code. An algorithm that moves binding across bundling as if these expressions were interchangeable changes the represented object. [A04]

This observation is more constructive than a general complaint that the representation is ad hoc. It suggests an operator-law test suite that runs before downstream reasoning experiments. Such a suite would not demand that all backends be groups or that all operations distribute. It would record which laws actually hold, under which operand types and approximation regimes. A backend with non-distributive binding might still be useful, provided the compiler never applies a distributive rewrite to it.

There is also a statistical issue. The normalised numerical similarity of two independent random byte coordinates has an expected value near 0.665. A coordinate near the centre of the range, such as 128, has a higher expected similarity to a random coordinate, near 0.749. The audit's 1,000-seed probe reproduced this difference. A bundle produced by averaging can move toward the centre and therefore look more similar to unrelated atoms even when it contains no useful association. [A04]

The result is not that a value near 0.75 is always meaningless. It is that the null distribution depends on what kind of object produced the score. Comparing two atoms, comparing an atom with a mean bundle, and taking the maximum over many chunks are different statistical tests. A single global threshold obscures those differences. If a system interprets all of them as a common notion of evidential support, its confidence can increase simply because a representation became more averaged or because more comparisons were tried.

These elementary counterexamples motivate semantic operator contracts and conditional calibration. A language system needs to know not merely vector size, but which algebraic laws and statistical assumptions its operations satisfy. This is a concrete reason to prefer explicit typed composition over an unrestricted bind-and-compare interface.

## 4. Growing representations and explicit storage

VSARepresentations contains several attempts to escape the capacity limits of a fixed holographic vector by changing how memory grows. Sparse, elastic, and exact backends trade fixed-size approximation for additional state, chunks, or explicit terms. The experiments are interesting because they make the resource trade-off visible: preserving more structure generally requires more memory or deferred computation. No tested representation yet provides both unrestricted structural fidelity and bounded fixed-size storage.

### 4.1 Sparse polynomial binding and the limits of cancellation

One of the most interesting aspects of VSARepresentations is the willingness to change the carrier rather than repeatedly tune one fixed hypervector. Its sparse-polynomial backend represents an object as a set of integer terms. Binding constructs pairwise XOR combinations, and sparsification retains a bounded number of terms. This can resemble an algebraic alternative to dense holographic superposition: preserve explicit sparse structure while still supporting cheap bitwise operations. [R03]

The implementation, however, must be distinguished from nearby mathematical constructions. A set removes duplicate elements. A polynomial over a field with characteristic two cancels duplicate contributions in pairs. These are different operations. If a design argument assumes cancellation while the code merely deduplicates, a claimed inverse may fail before any approximation or truncation is introduced.

A small probe makes the difference visible. Let the first sparse set contain the term 1, and the second contain 2 and 4. Full pairwise-XOR binding produces the set containing 3 and 5. Bind that result with the second set again. The four pairwise outcomes are 1, 7, 7, and 1. Set semantics retain 1 and 7. They do not cancel both occurrences of 7 or both occurrences of 1. The resulting set is not the original singleton. The audit used the backend's full binding operation, so the failure cannot be blamed on its normal sparsification step. [A04]

This does not establish that sparse symbolic carriers are a bad idea. It establishes that a proposed inverse requires a different algebra or a different claim. One could use coefficients and parity cancellation, restrict the binding keys, retain a richer factorisation structure, or define unbinding as a candidate-producing quotient rather than an inverse. Each repair has consequences for capacity, storage, and composition. The useful research question begins where the slogan “XOR is self-inverse” stops being sufficient.

The backend also places explicit limits on work and representation size. Its Cartesian products can be truncated, and its sparse result is reduced to a chosen number of terms. These choices are legitimate engineering decisions for bounded computation. But after truncation, exact reconstruction is no longer guaranteed even if the untruncated algebra were correct. A complete account must separate an algebraic failure, an approximation introduced for speed, and a recovery failure due to an inadequate decoder. The repositories sometimes move between these explanations too easily.

An archived design discussion provides a particularly revealing control. It reports that a configuration reduced to one small hash-like term could preserve strong symbolic evaluation results while HDC retrieval performance remained poor. That historical report

was not independently reproduced as a dedicated configuration in this audit. Nevertheless, the new multi-backend evaluation supports the broader concern: high final correctness can be maintained by a symbolic wrapper even when the approximate representation contributes little to finding exact answer sets. [R03; A03]

The correct lesson is not to discard the wrapper. The wrapper may be the useful part. It is to distinguish the architecture's success from the carrier's contribution. If a one-term identifier and a large holographic structure produce comparable accepted answers because the exact rule engine does the work, the expensive carrier has not yet justified its cost. Conversely, if the carrier reduces the number of exact operations needed, that reduction may be valuable even when its standalone accuracy is low. The experiment must expose those quantities separately.

Sparse representations therefore contribute two useful ideas to the future programme. They make growth and truncation visible, and they make it possible to discuss symbolic structure at a finer granularity than one dense memory vector. What they have not established is a cost-free reversible algebra for arbitrary composition. The next stage should treat sparsity as a resource policy over an explicit structure, not as evidence that every composite remains invertible.

## 4.2 Elastic metric-affine memory and explicit growth

The metric-affine-elastic backend addresses a different problem. Repeatedly averaging a growing collection into one fixed summary can make individual contributions difficult to retrieve. The elastic representation keeps bounded-size chunks, each with coordinate sums, a count, and a mean. Additional chunks are allocated as the



collection grows. A top-level summary can still provide a compact view, but it is no longer the entire stored state. [R03]

This is a significant conceptual change. A memory can look holographic at the level of each chunk while behaving like a segmented data structure at the level of the whole collection. Capacity is increased by allocating more storage, not by proving that one fixed vector holds unlimited information. The approach may be exactly what a practical system needs, but it must be described honestly. Its achievement is controlled growth with retained local statistics.

The audit measured the typed-array payload for a small configuration with dimension 32 and a chunk capacity of 32 items. One chunk required 192 bytes in the measured representation. At 33 items, allocation of a second chunk increased the payload to 352 bytes. A third chunk at 65 items brought it to 512 bytes; four chunks at 128 items used 672 bytes. These figures exclude JavaScript object overhead and other runtime structures. They are not complete process-memory measurements. They do demonstrate that a nominal 32-byte atomic vector is not an adequate description of a bundled elastic memory's cost. [A04]

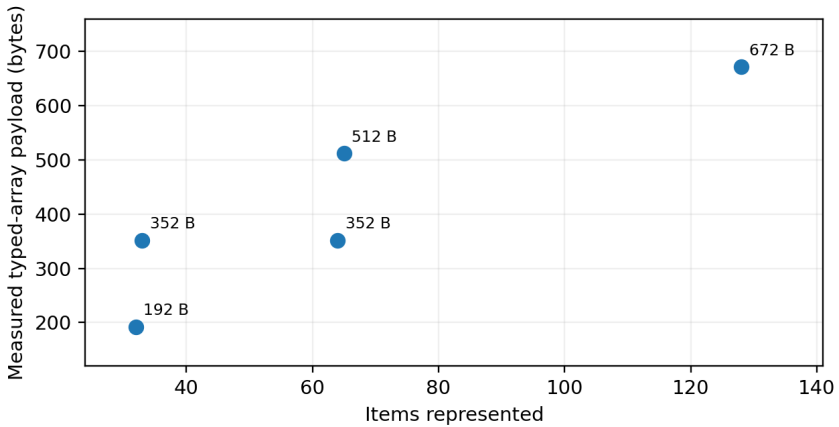


Figure 1. Elastic storage grows through chunk allocation. These are measured typed-array payloads for dimension 32 and chunk capacity 32, not full runtime memory. Each point is an executed probe. Source: A04.

The stored coordinate sums matter. If only an already-rounded mean were retained, subsequent updates would repeatedly compound rounding and weighting errors. Keeping sums and counts permits correct weighted aggregation at that level. This is a concrete positive engineering idea: distinguish a display summary from the sufficient state required for future updates. In a language memory, the analogous distinction is between a concise summary and the underlying records needed to revise it when new evidence arrives.

Elasticity does not automatically repair all operator problems. The inspected binding operation can apply XOR to chunk means and reconstruct sums from those means and counts. Information about the original distribution within a chunk is then absent. Combining bundles can also create Cartesian combinations of chunks. Furthermore, comparing against the best of several chunks changes the chance of seeing a large similarity by accident. A larger memory can improve recall while simultaneously increasing false

positives unless the acceptance procedure accounts for the number and type of comparisons.

The repository also supports deterministic atom generation that preserves a prefix when a larger dimension is requested. The audit confirmed that an atom's first 32 coordinates at dimension 64 matched its dimension-32 encoding. This is a useful compatibility property. It does not mean that a previously stored 32-coordinate bundle can magically reveal the missing 32 coordinates. To expand an old representation faithfully, the system needs the source atoms, sufficient construction records, or another way to recompute the extension. Prefix stability solves naming across dimensions, not recovery of unstored information. [A04]

There are therefore several distinct meanings of “growing representation.” The width of each vector may increase. The number of chunks may increase. The vocabulary of symbolic atoms may increase. The number of explicit combinations may increase. These growth modes preserve different information and incur different costs. Treating them as one property prevents a meaningful comparison. An experiment should specify exactly what is allocated, which earlier information remains accessible, and which operations become more expensive.

The interesting future contribution is an allocation policy driven by task obligations. A memory might create a new chunk when interference threatens retrieval, expand a structured record when a new role is introduced, or retain a source witness when a summary becomes ambiguous. Such a policy would connect storage growth to the failure it is meant to prevent. The elastic backend supplies useful building blocks, but this obligation-driven policy is a proposal of the present analysis rather than a result already demonstrated by the repository.

### 4.3 Exact monomial storage: replacing noise with combinatorial growth

The EXACT backend takes the move away from a fixed-size holographic state further. It maintains a session-local dictionary of atoms and represents a term through a set of atom bits. A bundle contains explicit terms. Binding forms pairwise unions of atom sets. This makes certain kinds of structural information exact rather than recoverable only through similarity. [R03]

The semantics are not those of an invertible group. Binding a term with itself does not duplicate its atoms, because set union is idempotent. The audit confirmed that binding a single atom with itself returns the same term. Removing that atom through the implemented quotient-like operation can produce an empty residual rather than reconstructing a second copy. This is expected under the chosen set semantics. It should not be described as a bug merely because it differs from a bipolar vector's self-inverse binding. [A04]

This distinction is important for a generic backend interface. An inverse answers, “Which unique input produced this result under a reversible transformation?” A quotient-like operation may instead answer, “Which residual structures are compatible with removing this query pattern?” The latter can be extremely useful for symbolic matching. It can also return multiple candidates or discard multiplicity information. A caller must know which question is being answered.

EXACT includes variants with different residual-selection behaviour. One construction gathers possible residual terms; another emphasises what remains common across query-related matches. These are meaningful alternative semantics for structural retrieval. They should be evaluated against corresponding tasks,

such as existential pattern completion or common-factor extraction, rather than forced under one label of “unbinding.” Their potential novelty lies in a useful representation-and-query combination with explicit guarantees, not in pretending that the entire carrier obeys one familiar VSA algebra.

Removing collision noise does not remove cost. Binding a bundle with two alternatives to another bundle with two alternatives can produce four combinations. Repeating the construction with independent alternatives yields exponential expansion. The audit's simple twelve-stage probe produced 4,096 explicit terms. That is the expected combinatorial growth, not evidence of an implementation error. An exact memory must either pay for such combinations, keep them factorised, or decline to enumerate them. [A04]

A factorised representation suggests a circuit directly. Instead of storing every expanded term, the system can retain a node saying that one bundle is bound to another. A query then chooses whether to expand, simplify, or match the structure lazily. This introduces a new burden: equality and retrieval may require computation over the factorised graph. The trade-off is no longer “noise versus no noise,” but “eager expansion versus deferred computation.” That is a more promising architectural question because it can be optimised according to actual tasks.

The backend's reported byte count also requires care. An accessor returning zero for an exact representation does not demonstrate zero storage; in context it is an unknown or non-applicable size sentinel. The atom dictionary, BigInt values, term collection, and runtime objects all occupy memory. Scientific reporting should not allow a convenience API to become a capacity claim. The same principle applies to the elastic backend's atomic width and to later neural active-parameter counts: a convenient component measure is not the cost of the whole system.

A fixed finite memory cannot assign a distinct code to every member of an unbounded set of structures. That follows from counting, not from a special weakness of VSA. Allowing storage, computation, or external cleanup memory to grow changes the premise. Published capacity analyses likewise formulate guarantees for specified tasks and representation families, including connections with sketching and Bloom filters; they do not provide an unrestricted capacity claim for any object called a hypervector. [L03] The exact backend is interesting precisely because it makes one such change explicit. The programme is not discovering that intelligence requires abandoning mathematics; it is discovering that storage growth, factorisation, and query execution must be part of the mathematical object. This is the strongest bridge from representation experiments to SOP circuits in the supplied collection.

## 5. Interference-aware memory placement

VSANash keeps the underlying VSA/HRR operations largely fixed and instead changes where memories are stored. The hypothesis is that interference can be reduced by retrieval-aware sharding and placement. This is one of the clearest positive systems results in the collection: under a low-load rerun, an interference-aware placement policy substantially improved retrieval relative to a simple hash placement, while high-load and multi-hop cases exposed the remaining limits. The result suggests that memory management may matter as much as the choice of binding algebra.

### 5.1 The VSANash placement mechanism

VSANash asks a better question than whether one memory vector can hold an arbitrarily large knowledge base. Given several bounded memories, where should facts be stored so that they remain retrievable? This makes interference a variable the system can influence. Instead of accepting a random assignment of facts to superpositions, the implementation searches for placements with more favourable retrieval behaviour. [R04]

The evaluated HRR configuration represents associations through distributed keys and values. A query retrieves a noisy estimate, and cleanup compares that estimate with a vocabulary of possible answers. When many associations share a memory, unrelated contributions can survive the retrieval operation and compete with the intended answer. A shard is a separate memory in which only some associations are superposed. Sharding reduces load, but then the system must route a query to useful shards without scanning everything.

The repository provides several ways to allocate facts. Simple policies hash a subject, a relation, or a combined key. Other policies try to reduce collisions or pack associations according to a heuristic. The equilibrium-style policy repeatedly considers moving an association and scores possible placements using retrieval-related quantities and load. Its purpose is to improve the memory from which that association will later be recovered. Two-write variants allow replication into more than one shard.

This is a concrete algorithmic idea, not merely an analogy with games. The code has a state, candidate moves, a utility calculation, and stopping conditions. However, the name “equilibrium” must not be allowed to imply a theorem not supplied by the implementation. A bounded sequence of best-response-like moves with a tolerance and a maximum pass count is a heuristic optimisation procedure. The audit did not establish existence, uniqueness, or convergence to a Nash equilibrium for the relevant objective. The implemented mechanism is interesting without those stronger claims.

The distinction between placement and routing is central. A fact may have been placed in a shard where it is recoverable, but the query may never visit that shard. Conversely, the router may select the right shard while the answer is lost among cleanup competitors. VSANash records route-hit information separately from answer metrics. That separation is valuable because it exposes different bottlenecks instead of compressing them into one accuracy number. The next research iteration should make the decomposition even more explicit at the per-query level.

An AI-attributed review already identifies that need. The supplied `antigravity_review.md` asks for per-query outcomes, margin decomposition, and broader scaling scenarios. It explains that aggregates cannot reliably distinguish routing failure, cleanup



ambiguity, and multi-hop noise accumulation. This is documented evidence of an earlier AI contributing an appropriate diagnostic direction. It is not evidence that the requested export or scaling work had already been completed. [R04]

The potentially innovative coupling is between storage decisions and downstream retrieval interference. General sharding, hashing, replication, and local search are established ideas. The research value here would come from showing that a VSA-specific retrieval signal provides a consistently better placement objective under fair resource constraints. A persuasive paper would compare it with equally budgeted alternatives, include insertion and update costs, and explain when the policy stops helping. The repository gives a concrete starting point for that investigation.

## 5.2 Observed low-load gain and high-load failure

The audit reran three scenarios with the HRR backend. Each evaluated eleven strategies on 300 queries. The vector dimension was 512, and the sharded configurations used eight shards. The scenarios contained 1,200 facts at low load, 6,000 facts at higher load, and a 5,000-fact two-hop task. These are single protocol configurations, not a dimension sweep or a statistical claim over independent worlds. [A03]

In the low-load case, the equilibrium-style strategy achieved a hit-at-K score of 70.0 percent. The key-hash baseline achieved 31.7 percent, and the collision-greedy strategy achieved 64.3 percent. This is a meaningful observed difference within the tested setup. It suggests that placement can change retrieval behaviour substantially without changing the underlying representation family. The comparison with collision-greedy is especially useful because it asks whether the more elaborate policy adds value beyond simply reacting to collisions.

Top-one accuracy gives a less triumphant picture. Both equilibrium-style and collision-greedy placement achieved 27.3 percent, whereas key hashing achieved 12.3 percent. The better memory often brought a correct answer into the candidate set without making it the highest-ranked answer. This is not a contradiction. It means that retrieval and final selection remain separate problems. A system with an exact downstream verifier might benefit from the wider candidate recall, but a system that directly emits the top candidate would still be wrong often.

Route-hit rate was about 95.3 percent for equilibrium-style placement in this low-load scenario, compared with about 82.7 percent for key hashing. Yet answer hit-at-K remained 70 percent and top-one accuracy much lower. Reaching a useful shard was not sufficient to recover the right answer reliably. This decomposition gives the next experiment a target: improve cleanup or verification rather than assuming the remaining error is all in routing.

The higher-load case sharply limits the conclusion. With 6,000 facts, equilibrium-style hit-at-K fell to 21.3 percent and top-one accuracy to 5.0 percent. Key hashing reached 10.0 percent and 2.0 percent respectively. The relative advantage remained, but the absolute result was weak. In the two-hop scenario, equilibrium-style hit-at-K was about 4.7 percent and top-one accuracy 1.0 percent. Collision-greedy placement did better there, at about 7.3 percent hit-at-K and 2.0 percent top-one. No claim that the heuristic solves multi-step reasoning follows from these results. [A03]

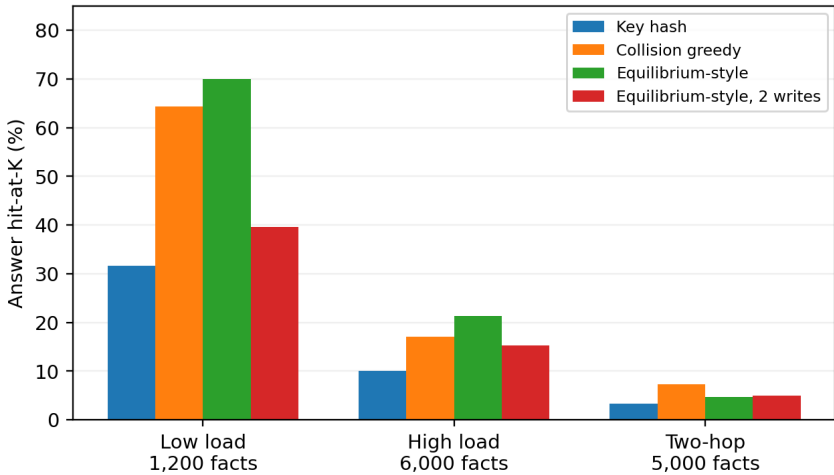


Figure 2. Retrieval gains are load- and task-dependent. Four equally sharded strategies, HRR dimension 512, eight shards, 300 queries per strategy and scenario. Hit-at-K is not top-one accuracy; no confidence interval over independent worlds is claimed. Source: A03.

Replication was not a free improvement. In the low-load case, the equilibrium two-write variant achieved about 39.7 percent hit-at-K, well below the single-write variant's 70 percent. Writing into more memories can improve the chance of visiting a copy, but with a fixed shard budget it also increases the amount superposed into those memories. The experiment makes that trade-off visible. Redundancy and interference move together unless extra storage or another control compensates for them.

The measured memory counters also require a fair reading. The single-memory configuration reported approximately 4.31 million bytes, while the eight-shard configurations reported approximately 4.42 million. These are implementation counters rather than complete runtime measurements and include a substantial common representation cost. A comparison between single and sharded memory is therefore not perfectly equal-memory merely because vector dimension is fixed. Comparisons among equally

sharded strategies are cleaner, although their build work and metadata may still differ.

The result worth preserving is neither “VSA memory scales” nor “VSA memory fails.” It is that inference-aware placement produces a useful low-load retrieval gain, but the gain does not remove overload or composition failure. That is a credible candidate mechanism with a visible operating region. The next study should vary load, vocabulary size, dimension, shard count, routing budget, and update frequency, then report where the policy improves total task cost rather than just one retrieval metric.

### **5.3 Research significance: interference-aware accountable memory**

The VSANash findings connect naturally with elastic and exact representations. All three approaches recognise that memory organisation matters. Elastic chunks allocate additional state. EXACT retains structure without collision noise. VSANash redistributes associations across bounded noisy stores. These are alternative responses to the same practical question: what should the system do when its current memory organisation no longer supports the required operation reliably?

A future architecture could make that response explicit. Suppose a query needs an answer with a source witness. A VSA shard returns several candidates but a poor margin. The system can search another shard, invoke an exact index, expand a factorised structure, or allocate a new memory partition for future queries of this type. The choice is not simply between “use VSA” and “do not use VSA.” It is between different ways to satisfy a retrieval obligation under a budget.

This suggests a useful distinction between storage health and epistemic confidence. A low cleanup margin is evidence that a retrieval mechanism may confuse candidates. It is not evidence that the underlying proposition is uncertain in the world. A high margin is evidence that the mechanism separates its current vocabulary well. It is not proof that the stored assertion is true. Mixing these two kinds of confidence would make a system appear more certain when a memory was reorganised, even though no new world evidence had arrived.

The same distinction applies to route-hit rate. A router can be excellent at locating a shard containing a false assertion. A provenance-aware reasoner must still decide whether that assertion is authoritative, current, and relevant to the requested world. Memory-placement optimisation should therefore preserve source identities and version boundaries. Repacking a memory should be a semantics-preserving maintenance operation, not a reason to promote a candidate to fact.

One concrete proposal is to attach a retrieval contract to each memory partition. The contract would state the supported key structure, the current load, observed error estimates on held-out probes, the exact source records represented, and the fallback available when recall is insufficient. Such metadata would allow a planner to choose a memory according to the task rather than rely on a single global similarity threshold. This is not implemented as a unified mechanism in the examined snapshots. It is an architectural extension suggested by their combined behaviour.

An equally important proposal is update-sensitive evaluation. The current gain could disappear if every new fact required expensive global reorganisation. A useful experiment would insert facts incrementally, measure movement costs, and test whether old associations remain recoverable. Another would delete or correct

assertions and verify that stale contributions do not continue to influence accepted answers. These tests connect memory efficiency with trustworthy operation: a memory that is fast but cannot reliably forget a withdrawn claim is not a good knowledge substrate for an agent.

The move toward SOP circuits would make these operations visible. Encoding, routing, candidate retrieval, source lookup, validation, and reorganisation could be separate nodes with different authority. Their outputs would not all be generic vectors. A candidate set would carry retrieval diagnostics; an accepted fact would carry a source or proof; a storage update would carry an invalidation record. The language model could then exploit VSA's associative strengths without confusing successful association with justified knowledge.

VSANash therefore deserves a place near the centre of the book. It contains an implemented, testable mechanism with a positive result and clear limitations. More importantly, it changes the architectural question from the capacity of one representation to the behaviour of a managed memory system. That change remains useful even if a later implementation replaces HRR with another approximate index.

## 6. Eos Gate: abductive property discovery and HDC decoding

The Eos Gate experiment in `vsaAbduction` asks whether a latent explanatory property can be inferred from examples and then used for prediction and intervention. Its strongest result is not open-ended scientific discovery; it is a reproducible pipeline connecting a restricted semantic hypothesis class to verified counterfactual repairs. A separate decoder audit performed for this book showed that an apparent HDC error could disappear when the readout was changed, demonstrating that some observed limitations belonged to the decoder rather than to the encoded information.

### 6.1 Intended experiment: discover an explanatory invariant

Eos Gate is the most self-contained experiment in the collection. It describes a synthetic world in which artifacts have attributes and combinations of artifacts either remain coherent or fracture. The learner sees descriptions and labelled event logs. The objective is to recover a latent additive property that explains those outcomes and can be used on held-out scenes. Unlike an unconstrained invitation to “discover a theory,” the experiment specifies a narrow hypothesis class and exposes the generator, learner, controls, and verification code. [R14]

The feature vocabulary contains twelve attributes: sunlit, moonlit, iron, glass, electric, organic, red, blue, ancient, silent, floating, and engraved. The generator assigns weights 2, -2, 1, -1, 1, and -1 to the first six respectively, and zero to the remaining six. An artifact's contribution is the sum of its attribute weights. A scene is coherent when its total contribution is zero. These weights are hidden from

the learner's intended input and used by the world generator and final checks. [R14]

The examples are concrete enough to reveal what generalisation means. The Helioforge Key is sunlit, iron, electric, and red, giving it a contribution of four under the hidden law. The Nulltide Lantern is moonlit, glass, organic, and blue, giving it minus four. One of each balances. But learning only that these two named artifacts balance would not explain a novel object with a new name and a familiar combination of attributes. The semantic representation makes that transfer possible.

The learner forms feature-count vectors for scenes and searches for a weight vector that vanishes on coherent examples. In the successful run, the coherent-scene matrix has rank eleven over twelve features, leaving a one-dimensional null space. The resulting weight vector matches the hidden law up to the conventional ambiguities of scale and sign, with the chosen normalisation recovering the reported coefficients. This is a restricted form of abduction: finding an explanatory invariant inside a predefined family of additive laws.

The audit regenerated the original experiment with seed 424242. It used 160 training scenes and three test splits of 300 scenes each: familiar artifacts, larger-count extrapolation, and novel artifacts. The semantic invariant classified all of these splits correctly. Thirty attempted counterfactual repairs were independently checked by the world rule and succeeded. A shuffled-label control no longer produced the intended invariant. These are rerun results, not merely statements copied from a saved summary. [A03]

The counterfactual step is particularly valuable. A classifier can separate positive and negative examples without giving a useful way to change a configuration. An additive invariant offers a residual: how far the scene is from balance under the learned



property. A repair procedure can search for an artifact change that cancels that residual, then ask the independent world generator whether the intervention works. This connects explanation, prediction, and action within one executable protocol.

The experiment's limits are equally important. Attribute extraction is controlled; the vocabulary is small; the world is generated from the same additive family the learner searches; and the observations are not arbitrary natural-language reports. The null-space calculation does not discover that conservation laws are the right hypothesis family from an unrestricted space of possibilities. It identifies a law after that representational commitment has been made. This is useful scientific evidence for the mechanism, but not evidence that a general language model has emerged.

The repository explicitly discloses coding-agent assistance and describes human conceptual iteration from anonymous symbols toward semantically described objects and counterfactual repair. That record supports a limited attribution: the experiment was shaped through human review and AI-assisted implementation, and its current scientific object is the executable protocol. It does not establish autonomous discovery by the coding agent. The improvement in experimental design is nevertheless real: the revised setup can distinguish memorised object identities from a property that transfers through attributes. [R14]

## **6.2 Observed result: separating representation from decoder error**

The symbolic abductor and the HDC representation must be separated. The abductor learns the twelve-dimensional weight vector from semantic features. The HDC stage encodes those features through a matrix of random distributed codes and uses a corresponding readout to estimate the learned quantity. If the HDC

stage performs poorly, that does not mean the invariant was not discovered. It means the distributed implementation of the already-discovered property needs examination.

Let  $x$  be the twelve feature counts in a scene and  $q$  the twelve learned weights. The desired score is  $q^T x$ . Let  $H$  contain the distributed feature codes as columns. The scene representation is  $Hx$ . A natural approximate readout is  $Hq$ ; comparing it with the scene and dividing by the vector dimension gives  $q^T(H^T H / D)x$ . This equals the desired score only when the codebook's normalised Gram matrix behaves like the identity on the relevant vectors.

Random columns are approximately orthogonal, not exactly orthogonal. Off-diagonal terms mix feature contributions. In a task where coherence means the exact zero level set of an additive quantity, small residual errors can matter. A larger dimension often reduces random correlations, but it does not make a finite codebook perfectly orthogonal. Increasing scene counts can also magnify the effect of remaining correlations. This gives a concrete reason that extrapolation may be more sensitive than familiar-count tests.

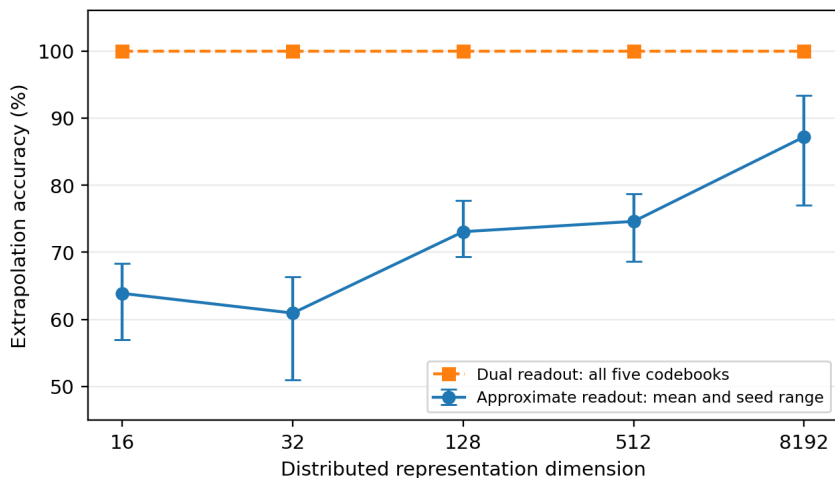
The key observation is that the encoding can still preserve all twelve feature dimensions. When  $H$  has full column rank, the mixed score can be corrected. Define  $G = H^T H / D$ , solve  $G a = q$ , and use  $Ha$  as the readout. Then  $(Ha)^T(Hx) / D$  equals  $q^T x$  in exact arithmetic. The correction is a dual-coordinate readout. It does not reconstruct the hidden law from labels; that law has already been learned. It changes how the known property is read from the distributed representation.

This is ordinary linear algebra, not a new theorem. It is also consistent with published practical VSA work that replaces similarity-based retrieval with linear readout in appropriate settings. The contribution of the present experiment is to apply the

correction to the actual Eos implementation and determine whether it explains the observed gap. A literature-aware account should credit the existing idea while reporting the new diagnosis of this codebase. [L06]

The audit reused the repository's world, parser, feature encoders, and abductor. It held the data seed fixed and varied the distributed codebook across five dimensions and five seeds per dimension. Both the original-style decoder and the corrected decoder fitted their decision thresholds using training data only. No held-out labels were used to calibrate the correction. The tested dimensions were 16, 32, 128, 512, and 8,192. All twenty-five sampled codebooks had full column rank. [A05]

The corrected readout achieved 100 percent classification on training and all three held-out splits in every sampled configuration. Its largest absolute discrepancy from the direct semantic score was below  $4.7 \times 10^{-13}$ . At dimension 8,192 and the first tested codebook seed, the approximate readout's extrapolation accuracy was 82 percent; the corrected readout achieved 100 percent on the same cases. Across the five 8,192-dimensional codebooks, the approximate extrapolation results ranged from 77 percent to about 93.3 percent. [A05]



**Figure 3. Changing the readout changes the interpretation of the representation.** Points show mean extrapolation accuracy across five codebooks per dimension; bars show the observed minimum-to-maximum range, not a confidence interval. The data world is fixed. All tested dual readouts score 100 percent. Source: A05.

This changes the explanation of the error. For these unquantised, full-rank encodings, the information needed for the property was present. The approximate decoder did not recover it accurately. The result therefore supports the human researcher's caution that engineering or operator design may account for some apparent VSA limitations. It does not overturn the broader concern about general reasoning, because the experiment still begins with twelve explicit semantic features and a learned additive law.

### 6.3 Research significance and claim boundary

The smallest tested representation had sixteen coordinates for twelve semantic features. It was not a compression of those twelve degrees of freedom into fewer independent measurements. The corrected readout also used floating-point coefficients rather than restricting itself to the original codebook's simple form. Its

numerical success should not be presented as a free gain in efficiency or as evidence that arbitrarily large symbolic structures can be recovered from tiny vectors.

The direct semantic baseline remains decisive. Once the system possesses  $x$  and  $q$ , computing the twelve-term dot product is simpler than encoding into thousands of coordinates and decoding again. The audit establishes that HDC need not lose this particular quantity under the tested conditions. It does not establish that HDC is useful for computing it. The representation might become useful if it supports additional operations, interoperates with noisy upstream signals, or shares hardware and memory with other tasks. Those advantages require separate experiments.

The correction also relies on a stable, known feature codebook. If the feature universe grows until the encoding becomes rank-deficient, the same inverse does not exist. If bundling is followed by hard quantisation, clipping, or nonlinear transformations, the linear identity no longer applies as written. If parsing omits the relevant feature or the true law is not in the additive hypothesis class, a perfect decoder cannot repair the error. Distinguishing these failure classes is the practical value of the analysis.

An additional caution concerns repeated seeds. Twenty-five codebooks provide evidence about representation sensitivity for one fixed synthetic data world. They are not twenty-five independently discovered scientific laws. The novel-object split tests new names and attribute combinations inside the fixed feature vocabulary, not unknown kinds of property. A future study should vary the law, feature basis, observational noise, and hypothesis class, with particular attention to cases where multiple explanations fit the training data.

Existing VSA abduction research also sets a clear boundary around novelty. ARLC, for example, combines programmed knowledge and

learned rules for abstract reasoning tasks. The generic phrase “VSA for abductive reasoning” is therefore not a novel contribution of Eos Gate. The relevant question is whether the particular combination of semantic descriptions, invariant identification, held-out transfer, and verified repair adds something useful beyond nearby approaches. That would require a direct comparison of task assumptions and mechanisms, not only a list of accuracy numbers. [L05]

The most promising continuation is to expand the hypothesis language while retaining the discipline of intervention. Instead of searching only for a single additive invariant, the system could consider piecewise laws, relational constraints, or small executable programs. Each candidate would explain observations, generate predictions, and propose repairs. A coding agent could construct candidate circuits, but acceptance would depend on held-out evidence and independent intervention checks. This is an explicit route from the existing experiment toward a circuit-centred research programme.

That route must avoid handing the answer to the learner through the generator. Candidate families should be declared, generator parameters isolated, and tests include laws outside the current language. Failure on an out-of-family law should produce a recognisable “not represented” outcome rather than forced confidence in the nearest available invariant. The system should also preserve competing laws when the evidence does not distinguish them. Here Eos Gate connects directly with autoResearchLib's frontier of alternatives.

The enduring insight is that explanatory knowledge can be operational: a property is valuable because it supports prediction and controlled change. HDC is one possible carrier for that property, and in this experiment its apparent weakness was partly

a decoder issue. The broader architecture should preserve the distinction between discovering a law, representing it, evaluating it, and trusting its interventions. Making those separate executable circuits is not a retreat from the experiment. It is a way to retain what the experiment actually demonstrated.

## 7. Structural representations for events and text

VSABrains2 and VSAText investigate how events, entities, roles, contexts, and narrative state might be represented for retrieval and reasoning. Their documentation aims at episodic memory and structured text processing rather than unrestricted text generation. The audit found both useful explicit structures and encoder collisions. The central lesson is that role identity and event direction must be preserved before approximate retrieval can be trusted to support reasoning.

### 7.1 Episodic memory requires role-preserving structure

VSABrains2 explores a structured answer to the memory problem. Its experiments organise entities, reference frames, locations, events, checkpoints, and transitions rather than relying exclusively on one undifferentiated superposition. Multiple columns can contribute estimates, while explicit episodic machinery reconstructs states from changes. This direction matters because many apparently linguistic questions are really questions about state and perspective. [R05]

Suppose an object moves from a shelf to a cupboard while one character is absent. The current location, the earlier location, and the absent character's belief are different queries. A memory that stores only the latest location cannot answer all three. A memory that stores both locations without time or perspective may confuse them. Increasing the dimension of an unordered encoding does not provide a missing state model. The representation needs to preserve the relation between an event, its participants, its time, and the view in which it is accepted.



The rerun second experiment in VSABrains2 used a small synthetic event sequence with structured queries. It reported perfect performance on its default state, coreference, motif, temporal, and conflict checks. The setup used 100 events and ten queries, with explicit transition rules and controlled schemas. The result supports the implemented state machinery in that world. It does not establish unrestricted natural-language extraction or arbitrary long-context reasoning. [A03]

That distinction is not a reason to dismiss the result. Exact state updates and event replay are useful components. In a language-capable system, an external parser or coding agent might construct the event representation, after which a deterministic engine maintains its consequences. The architecture then has two different accuracy questions: whether the event was extracted correctly, and whether the state update follows correctly from the extracted event. Separating them makes errors easier to diagnose and corrections easier to propagate.

The multi-column experiment also produced an apparent consensus advantage. Its first reported consensus figure was 80.4 percent compared with 70.1 percent for the best-column figure. Inspecting the aggregation code revealed that, although the scenario loop ran three trials, the returned consensus statistics were taken from the first matching trial record rather than a pooled three-trial summary. The numbers should therefore be read as the reported first-trial values, not a robust average. The observation narrows the evidence without eliminating the underlying consensus hypothesis. [R05; A03]

A corrected evaluation should expose each column's error pattern, not only its mean accuracy. Consensus helps when errors are sufficiently different and the combination rule uses that diversity. Multiple copies of the same mistaken transition logic can agree

perfectly and remain wrong. If columns share the same parser and state assumptions, a measured gain may be smaller on systematic ambiguities than on injected random noise. The programme should test disagreements caused by alternative interpretations, not only numerical perturbations.

The strongest architectural idea is that memory can be factored by function. A current-state cache answers one class of query quickly. An event log supports reconstruction. A perspective-specific view supports belief questions. A source record permits correction. VSA may index or associate these structures, but the structures determine what can be answered. A later SOP implementation could turn this factorisation into explicit circuits for event admission, state transition, snapshot construction, and retrospective query.

This would also make growth more meaningful. New events add records. New types of event add transition circuits. A new perspective adds a view over shared events rather than forcing every distinction into a larger global vector. Such an architecture gives up the elegance of a single homogeneous carrier, but it gains a direct account of why memory grows and what each new part enables. That trade-off is supported by the practical direction of the experiments, even though the complete unified architecture remains to be built.

## 7.2 Index quality versus semantic competence

One VSABrains2 experiment is especially instructive because the implementation is much simpler than its surrounding research theme might suggest. The sixth experiment, labelled as a literature-oriented evaluation, creates 100,000 synthetic records containing a frame, subject, and value. It builds a nested map from frame to value counts and answers forty queries about the most frequent

value. The comparison is between scanning the records and consulting the precomputed map. [R05]

The audit observed a very large query-time difference, but it is not used here as a VSA speed claim. The fast path is an ordinary exact index, the index-construction cost is outside the timed query phase, and the run occurred on shared hardware while other jobs could execute. The useful observation is structural: when the query family is known, an appropriate index can turn repeated scanning into a much cheaper lookup. Nothing in that observation requires holographic reasoning. [A03]

This matters because the programme is searching for CPU-efficient intelligence. It is tempting to associate an impressive speedup with the most distinctive technology in the project. But the effective mechanism may be a familiar data structure. That is good news for engineering and a warning for novelty attribution. A practical architecture should use the map when it solves the task, reserving VSA for cases where exact keys are unavailable, approximate association is useful, or the query structure cannot be anticipated so directly.

The same principle applies to the symbolic wrappers in VSARepresentations. Its fast evaluation ran twelve representation-and-reasoning configurations over 421 assertions per configuration. Several configurations achieved 421 accepted results, while others missed one or two. Some exact-symbolic modes attributed essentially none of their accepted answers to a final HDC method. Holographic modes sometimes reported many accepted HDC-labelled answers while the separately measured exact answer-set match on a small diagnostic subset remained zero. These counters measure different things and must not be collapsed into one statement about HDC accuracy. [A03]

An accepted answer may be one valid member of a larger answer set. Exact answer-set equality asks whether the retrieved set matches all and only the expected answers. A method label may indicate which branch returned the candidate, not that the branch supplied the decisive semantic justification. The evaluation is valuable precisely because it contains several counters, but their meanings must be preserved in reporting. Zero exact matches on fourteen or twenty selected cases is not zero accuracy across all 421 assertions. Conversely, 421 accepted assertions are not 421 demonstrations of successful purely holographic reasoning.

These examples suggest a more useful efficiency benchmark. Fix the semantic task and measure the entire path: extraction, index construction, retrieval, exact checks, updates, and memory maintenance. Compare cold-start and amortised workloads separately. Report the amount of exact work avoided, not just the speed of one query stage. If VSA reduces a million candidate checks to a few dozen, that is valuable. If a hash index reduces the same work further at lower memory cost, the architecture should prefer the hash index for that workload.

The innovation opportunity is not to replace every data structure with vectors. It is to select the right representation based on the query's operational requirements. A circuit planner can choose exact predicate indexes for known structures, associative search for weakly specified cues, and event replay for temporal questions. The choice becomes an explicit policy that can be measured and improved. This is a more credible route to efficient agentic systems than attributing all acceleration to one representation family.

### **7.3 Observed role-order collision in narrative encoding**

VSAText contains a substantial narrative-oriented system: schemas for characters and scenes, parsing and generation infrastructure,

structural metrics, and optional language-model interactions. Its design documents are often appropriately cautious about the role of VSA, treating it as retrieval support rather than a complete reasoner. The audit nevertheless found a sharp mismatch between what a generic text vector might be expected to preserve and what the default encoder actually preserves. [R06]

The encoder lowercases and tokenises text, generates deterministic word vectors, and bundles them. Its default `encodeText` path does not bind words to positions or semantic roles. The audit encoded “Alice helps Bob.” and “Bob helps Alice.” The resulting 10,000-component vectors were exactly equal, with cosine similarity one. This was not an approximate collision caused by insufficient dimension. The two sentences have the same token multiset, and the encoder’s operation is permutation-invariant. [A06]

There are tasks for which this is acceptable. Topic search may benefit from ignoring word order. A system retrieving passages about Alice, Bob, and helping could reasonably place both sentences together. But a system deciding who performed the action cannot rely on this representation alone. The distinction must be supplied by another structure or a different encoder. This is an example of a missing semantic contract, not a reason to reject every use of bag-based HDC text retrieval.

The repair is conceptually straightforward but operationally demanding. A parser could produce an event with an actor, action, and recipient. Role binding could then preserve the distinction between actor and recipient, while the canonical event record remains available for exact inspection. However, pronouns, passive voice, reported speech, and ambiguous attachments make extraction difficult. Adding role vectors does not solve parsing. It gives a downstream representation somewhere to put the distinctions once they are identified.

The audit also examined a metric named character drift. The tested function derived its drift score from the average number of listed traits. A character with the traits kind, honest, and patient received the same score as one with kind, cruel, and honest. Both had drift 0.05, score 0.8, and a passing outcome. The point is not that kind and cruel can never coexist in a complex character. It is that the metric does not inspect their semantic relationship or any change over time. It measures a structural proxy, not demonstrated temporal character consistency. [A06]

This finding should guide naming and evaluation. A trait-count sufficiency metric may be useful: it can warn that a character record is underspecified. Calling it a drift detector invites a different interpretation. A real drift task would require a baseline state, later evidence, contextual explanations for change, and a criterion distinguishing contradiction from development. An executable event-and-belief model could support that task far more directly than a renamed count.

A supplied Codex-attributed review identifies another gap: pattern-related structures were present without a complete runtime for the intended variable and template expansion behaviour. The review recommends either implementing the minimal runtime or narrowing the claim. This is a good example of useful AI criticism already present in the corpus. The 268 passing tests show that many implemented components work under their tests; they do not close the gap between a stored pattern description and an executable semantic transformation. [R06; A02]

The positive direction is a narrative memory in which prose is a view over explicit structures. Characters, locations, events, perspectives, and causal claims remain separately inspectable. Approximate text vectors help locate relevant records; they do not replace role and temporal identity. A coding agent may compile

text into those structures, and another circuit may generate a summary from them. The book's proposed SOP direction preserves this separation. It does not assume that structured data alone creates literary intelligence, but it gives explanation and correction a concrete object on which to operate.

## 8. Prediction, transformation, and executable compression

SomaVM, BSP, and VSAVM explore a different route toward language competence: prediction and compression as executable transformations rather than as a single global holographic memory. The implementations range from local integer learning to explicit bitset transforms and macro-like executable units. Their results are mixed, including weak predictive checkpoints and an evaluation metric that can award a perfect negation score to a system that produces no answer. The enduring idea is that reusable transformations may be better represented as explicit executable objects than as implicit vector equations.

### 8.1 SomaVM: local predictive learning

SomaVM is an attempt to make a continuously adapting predictive machine from bounded integer operations. It consumes bytes, predicts the next byte, and can produce output. Its model includes integer embeddings, transformer-like hard attention, ternary projections, clipping, and local updates with eligibility traces. An internal reward comes from prediction; an external signed signal can modify the combined reward. The terminology in the repository includes “pleasure,” but the implemented object is a scalar learning signal. [R08]

The attraction is practical and conceptual. A small deterministic machine with local updates might be cheaper to inspect and operate than a conventional large neural model. Local learning also offers a possible route to continuous adaptation without a full global backward pass. However, the existence of a local update rule does not establish that it assigns credit to the right causes. A next-



byte error can arise from an embedding, an attention choice, a context limitation, or a deficient output rule. A reward broadcast through bounded eligibility traces must somehow distinguish useful changes from reinforcing incidental correlations.

The repository's seven tests passed in this audit. That supports the mechanics they exercise, not language competence. To obtain a more substantive observation, I evaluated a supplied checkpoint on its included question-answer dataset. The first attempt exposed a configuration dependency: the evaluator constructed the default model shape before loading a checkpoint with different dimensions. Supplying the checkpoint's own configuration allowed the evaluation to run. This is another example of an engineering condition that should be separated from model quality. [A02, A03]

The evaluated checkpoint used model dimension 128, attention-head dimension 64, and context length 64. Its episodic memory and prompt-memory options were disabled. Those details matter because a claim about the result should not be generalised to every memory-enabled variant in the repository. The evaluation file contained twenty records, despite the command permitting a maximum of one hundred. File names and command limits are not sample counts.

On these twenty included records, exact-match rate was zero and mean F1 was approximately 0.0472. The reported next-prediction accuracy was approximately 0.0556. The questions included simple paraphrases such as asking the colour of the sky or the sound made by an animal. These results do not support useful question answering for this checkpoint. They also do not establish a general impossibility of integer predictive models or local learning. They establish a weak result for a specific supplied state and configuration. [A03]

The most useful interpretation is that the programme has implemented a learning substrate without yet demonstrating the desired learned behaviour. That distinction should guide the next experiment. Before increasing the corpus or adding more machinery, the system should pass progressively harder controlled tasks: exact repetition, delayed copying, controlled variable substitution, short compositional transformations, and transfer to paraphrased prompts. Each task isolates a different demand on memory and credit assignment.

A good local-learning study would also compare the update rule with simpler alternatives on the same state budget. A frequency table or small n-gram model may outperform a more elaborate predictive VM on some workloads. A conventional small model may learn the same task with fewer examples. Such results would not make the VM useless; they would identify whether its advantage lies in adaptation cost, bounded arithmetic, interpretability, or some other property. Without a comparison, architectural simplicity is an aspiration rather than an empirical benefit.

SomaVM contributes an important research question even in its present state: can local predictive adaptation be attached to an explicit memory-and-program architecture rather than expected to invent the whole architecture implicitly? A circuit could decide what context to preserve, what prediction task is active, and when an update is accepted. The learned component could then solve a bounded subproblem. This proposed division would make it easier to discover whether failure comes from missing state, inadequate learning, or an unsuitable objective.

## 8.2 BSP: explicit transforms and a defective negation metric

BSP approaches prediction through explicit groups, counters, and compression operations. It is not simply a standard random-hypervector language model. Its structures increasingly resemble symbolic dictionaries and small programs: copying from context, emitting literals, repeating patterns, and representing reusable groups. That movement away from one holographic carrier is itself informative. The system finds useful regularity by exposing operations that can reconstruct sequences. [R09]

Archived experiments report gains from copy operations and from decoupling vocabulary-related costs. In one report, a groups-only configuration had an estimated cost near 2.98 bits per character, compared with about 2.20 when mechanisms were combined. Another report gives a lower value and compares it with gzip. These are historical modeled-cost results, not independently reproduced end-to-end compressed-file sizes in this audit. The distinction matters because a compression cost model can omit the storage needed for a vocabulary, program dictionary, context, or actual entropy coder. [R09]

A model that reconstructs text through a short program is nevertheless a useful direction. The program can reveal whether a gain came from copying an earlier substring, using a frequent literal, or reusing a structural pattern. This is more explanatory than reporting a single prediction loss without identifying the mechanism. It also connects naturally with the idea of learned SOP circuits: a reusable transformation becomes an explicit object that can be inspected and composed.

The actual implementation does not support every operation suggested by its design documents. For example, a template encoding branch in the compression machine is disabled in the inspected source. The presence of a template concept in a

specification or operation type is not evidence that the active compressor uses it. This is a recurring pattern in AI-assisted prototypes: a broad design vocabulary is implemented incrementally, while the documentation can make the incomplete system look more general than its executable path. [R09]

The abstract-primitives evaluation gives a particularly sharp warning. A bounded audit run completed six systems before timing out during the seventh. Convergence scored 1.2 percent, divergence 29 percent, cycles zero, hierarchy zero, composition zero, and negation 100 percent under that runner's criteria. Because the full sweep did not finish, these are partial results, not a completed benchmark over every proposed primitive. [A03]

The perfect negation result initially looked like the one strong capability in a weak set. Inspecting the scorer changed the conclusion. Its correctness condition required the forbidden answer not to appear among predicted candidates. It did not require a suitable allowed answer to be returned. An independent audit probe generated 1,000 test cases and passed them to an untrained BSPEngine. Every candidate list was empty, and all 1,000 cases received a correct score. [A06]

This is a logically valid test of exclusion but an invalid standalone test of useful negation. A system that says nothing can avoid every forbidden token. A repaired benchmark must demand positive coverage: return an admissible answer, reject specified inadmissible answers, and distinguish unsupported from contradicted propositions. It should also include prompts whose correct behaviour is abstention, so that forcing an answer everywhere does not create the opposite pathology. Negation is a semantic operation with several possible task forms, not the absence of a token in an empty set.

The result has a broader implication for evaluating efficient agents. Refusal, silence, or failure to act can appear safe under a metric that only counts harmful outputs. A useful system needs both safety and task completion, with explicit handling of justified abstention. The BSP scorer is a small, reproducible example of that general evaluation problem. Its correction may be more valuable to the programme than the original perfect number.

The compression reports also invite a mathematical caution. A unigram character entropy is not the entropy rate of a structured source and is not a universal lower bound on a compressor using context. Conversely, an estimated operation cost below a comparator does not prove a new information-theoretic limit. The credible research path is to specify an actual decoder and account for all transmitted or shared state. What makes BSP interesting is explicit program-based reuse, not an unsupported claim that a cost model has defeated a general entropy bound.

### **8.3 VSAVM and the move toward executable transformations**

VSAVM explores another route toward language behaviour: a virtual machine with structured units, memory, and a macro-oriented model for continuation. It also contains evaluation categories and optional retrieval or neural-related paths. This makes it an architectural experiment rather than a single learned predictor. The question is whether reusable execution structure can serve both prediction and semantic tasks. [R07]

The audit found a useful contrast between its evaluation and test suites. The standard evaluation reported all eleven categories successful. The unit suite still failed on two generation cases because the code accessed `budgetMs` before initialisation. After moving the declaration, the complete selected unit suite passed 307

tests. Both observations are true. They concern different execution paths and obligations. An aggregate “all evaluations passed” statement would have hidden an actual inability to execute two tested generation calls. [A02, A03, A07]

This is a direct demonstration of why the human's engineering caveat matters. A failed continuation can be caused by the runtime before the predictive mechanism gets a chance to operate. A repair that removes the exception is scientifically useful because it rules out that cause for subsequent experiments. It does not establish that the generated text is good, that the model generalises, or that the macro representation is sufficient. Those are separate evaluations.

The more interesting hypothesis is that learned macros could become semantic circuits rather than only frequent byte fragments. A byte-level macro captures a recurring surface sequence. A semantic macro might capture an event transition, a proof pattern, a summarisation decision, or a parameterised explanation. The latter can transfer across different names and wordings because its variables and preconditions are explicit. But that transfer must be tested; renaming a repeated substring as a “concept” does not create it.

A decisive experiment would present the same operation through multiple surface forms and then withhold a combination of arguments. The learner would need to construct or select a reusable circuit whose execution gives the right result on the held-out combination. The comparison should include literal memorisation, byte macros, an exact symbolic template system, and the proposed induced circuit system. Compression of training examples alone would not suffice. The circuit must earn its place by supporting a new operation or a more efficient execution.

BSP's copy instructions and VSAVM's macro machinery offer parts of that path. CNL's explicit syntax and typed structures offer another part. The programme need not insist that all reuse first appear as vector arithmetic. It can represent a reusable object directly and use VSA to retrieve it when an approximate cue is available. This avoids asking the approximate representation to be simultaneously a dictionary, a program, a proof, and a source of linguistic fluency.

The next architecture should also distinguish long-term learned units from task-local constructions. A macro generated to answer one prompt may rely on assumptions that should not enter the global knowledge base. Promoting it should require evidence of reuse, explicit dependencies, and tests outside its construction context. Invalidation should be possible when a source fact or operator contract changes. These are natural circuit-management operations, but they are not automatically provided by storing a macro in a library.

The lesson of the three prediction repositories is not that prediction should be abandoned. It is that prediction becomes more accountable when the system can expose what structure it reused and what operation it performed. Local learning, byte continuation, and compression remain useful objectives. Their strongest role may be to discover or select explicit computational units, while exact execution and validation prevent a successful surface pattern from being mistaken for general knowledge.

## 9. Exact reasoning and holographic retrieval

UBHNL, CNL, and HoloProof move the research programme toward typed representations, constrained language, explicit proof, and solver orchestration. The intended role of VSA shifts from being the authority for truth to being a retrieval or proposal mechanism. This separation is important for explainability and trustworthy AI. The audit demonstrates a concrete scope problem: a correct proof over a retrieved subset does not establish a conclusion about the complete knowledge base unless retrieval coverage is also part of the proof contract.

### 9.1 Typed representations as semantic control

CNL and UBHNL explore explicit semantic structure through constrained language and typed internal representations. This is sometimes described as a retreat from a more ambitious vector model. The code suggests a different interpretation. These systems make the objects of reasoning inspectable: declarations, predicates, arguments, constraints, rules, and execution budgets. That is a substantive architectural capability, not merely an emergency fallback when similarity is low. [R10, R11]

CNL implements a compilation-oriented session. New source is learned into a candidate state, the relevant sources are compiled, and the update is committed only if compilation succeeds. This makes malformed or inconsistent-with-the-language input a controlled failure rather than a partially applied change. The mechanism does not guarantee that accepted source describes the real world correctly. It does provide a meaningful transactional boundary for the representation the engine can execute. [R11]



The distinction between syntactic admission and truth is important. A statement can be well typed and false. A rule can be executable and inappropriate for a domain. A transaction can preserve internal invariants while accepting a misleading source. Nevertheless, clear stages make stronger checks possible. A future circuit can require source validation before promotion, or require a proof that a proposed rewrite preserves a specific result. Without an explicit intermediate object, such checks are much harder to state.

CNL's inspected implementation also contains provenance-related machinery, numeric and bitset-oriented representations, rule processing, and bounded planning. Its session limits include explicit depth and node budgets rather than an unqualified promise of arbitrary search. The audit passed 85 tests and eleven smoke suites. This is evidence of an implemented constrained language and its tested operations, not evidence that unrestricted English can be translated losslessly into it. [A02, A03]

UBHNL is at an earlier stage. It contains typed ASTs, frontends, a canonicalisation-oriented kernel, and design work toward a broader reasoning system. Its default development test command stated that no development tests had yet been implemented, while the separate typed-AST checks passed. The entry path principally parses and exposes structured forms; the wider solver and planning ambitions in its specifications should not be presented as completed capabilities. [R10; A02]

This difference in maturity is scientifically useful. It shows why a repository should be described by executable obligations rather than by the breadth of its design documents. “Can parse this typed fragment and produce a canonical AST” is a smaller claim than “can implement universal reasoning,” but it is a claim another researcher can test. A programme can build reliably from such

claims, whereas broad undocumented gaps make progress difficult to measure.

The positive idea shared by these projects is that natural language can be treated as a surface for explicit semantic objects. A human or coding agent may construct those objects from documents, but the resulting representation can be inspected independently of the prose that produced it. The system can then maintain alternatives when translation is ambiguous, instead of forcing a single interpretation and pretending the uncertainty disappeared.

This is a plausible foundation for SOP Lang circuits. The circuit is not required to replace every logical formalism. It can call an appropriate evaluator over a typed fragment while preserving the translation, scope, and evidence that justify the call. The novelty of the future system would lie in how these executable objects are composed, revised, and reused. Typed ASTs and constrained languages themselves are established tools, and their presence should not be marketed as a new discovery.

## **9.2 HoloProof: separating retrieval from formal verification**

HoloProof makes the hybrid architecture explicit. It has formal proposals, a world registry, accepted fragments, solver adapters, optional VSA intuition, query execution, and response-oriented components. The intended division is attractive: an approximate component can help choose promising material, while a formal solver checks the actual obligation. This is the right direction for reducing the authority granted to an opaque similarity score. [R12]

The repository also contains scenario and world-management ideas that are valuable for agents. A hypothetical world should not silently overwrite the accepted one. A proposed assertion should not become trusted merely because it was generated fluently. A

response should be connected to formal evidence and provenance. These are good architectural boundaries. Their effectiveness, however, depends on the details of the control flow around the solver.

The audit's deployment observations were mixed in a way that must not be obscured. All 73 selected tests passed. The separate smoke evaluation attempted twelve combinations of cases, solver choices, and intuition modes, but all twelve produced errors because the external solver binaries were unavailable. The evaluation process itself exited successfully. A process exit code of zero therefore did not mean that the reasoning cases passed. The unit tests largely exercised controlled adapters and mocks, not a live end-to-end solver installation. [A02, A03]

Rather than infer anything about solver quality from missing executables, I examined the query orchestration directly. For entailment, the code selects an active subset of fragments, asserts them, negates the goal, and asks whether the result is satisfiable. If unsatisfiable, the selected premises entail the goal under the chosen logic. If satisfiable, they do not. The crucial issue is what happens when those premises are only a retrieved part of the available world.

The implementation stops expansion on a definite solver verdict. Expansion is principally associated with an unknown result, not with a satisfiable counterexample obtained from an incomplete slice. This is understandable as a local solver protocol, but dangerous as a whole-knowledge-base protocol. A definitive answer about a subset is not necessarily definitive about the full set. The solver can be perfectly correct while the architecture assigns its answer the wrong scope.

To test this without claiming a live SMT deployment, I wrote a small independent Boolean adapter that enumerates all four assignments

of two variables. It parses the assertions emitted by the real query executor and evaluates their Boolean content. It does not return a prearranged sequence of verdicts. It is exact for the tiny Boolean language used in the probe, not a general SMT implementation. This makes it possible to test the actual orchestration with a transparent solver. [A06]

With available fragments  $q$  and  $p$ , goal  $p$ , and a one-fragment expansion step, ranking  $q$  first caused the executor to return not-entailed after one round. Ranking  $p$  first returned entailed. Including both immediately also returned entailed. The first result is correct for the active slice containing only  $q$ , but it is not a correct whole-world non-entailment conclusion because  $p$  is present in the available world. The probe exposes a scope problem, not a failure of Boolean reasoning. [A06]

The repository's knowledge-gap diagnostics partially recognise this first case: they note that the active slice does not support the goal symbol. That is useful, and a higher-level response policy may choose to ask a question rather than make an unconditional negative claim. The audit therefore does not assert that every user-facing answer is false. It establishes a lower-level contract hazard: a not-entailed label can be produced after an incomplete search unless the scope travels with it.

### 9.3 Proof validity requires retrieval-scope evidence

A stronger probe removes the simplest lexical warning. The first fragment is the tautology  $p$  or not  $p$ , and the second fragment is the fact  $p$ . The goal remains  $p$ . The first fragment mentions the relevant symbol, so the symbol-occurrence-based gap detector reports no gap. Yet the tautology alone does not entail  $p$ ; it is compatible with either truth value. The executor returns not-entailed after its first round, even though the second available fragment would establish

the goal. The audit reproduced this using the same independent Boolean adapter and the actual orchestration code. [A06]

This is a compact result with broad architectural significance. Mentioning a symbol is not evidence that all relevant information about it has been retrieved. A retrieved paragraph can discuss a topic without containing its decisive fact. A solver can see every variable name and still lack the premise that changes the answer. Therefore a lexical knowledge-gap check cannot serve as a completeness certificate for reasoning over a larger corpus.

The underlying logic explains the asymmetry. If a subset of premises entails a proposition in classical monotonic logic, adding more premises preserves entailment, although inconsistency introduces separate concerns about what conclusions should be accepted. By contrast, failure to entail from a subset says little about a larger set. Additional premises can establish the proposition. Similarly, a model satisfying a slice may violate omitted constraints. Negative or consistency conclusions often require a stronger account of coverage than a local positive proof.

A corrected architecture should represent at least the difference between a local result, a globally justified result, and an unresolved search. The exact names are less important than the contract. A result should identify the premises actually checked, the world version, the search policy, omitted regions, and the condition under which the verdict can be promoted. A budget exhaustion should not be silently converted into an answer about all available knowledge.

This does not require exhaustive retrieval in every case. An exact predicate index may certify that all rules with a relevant head were considered. A dependency analysis may prove that omitted fragments cannot affect the query. A finite domain may permit a complete enumeration. A user may explicitly ask about one

selected passage. The point is that completeness is a property to establish or limit, not an implication of running a trusted solver.

The HoloProof result also changes how explainability should be understood. A trace saying “the solver returned SAT” is technically accurate but insufficient. The useful explanation is that the selected premises admitted a countermodel, while additional available evidence was not considered. A trustworthy trace must show the boundary of the computation, not merely the name of the component that produced the answer.

The proposed SOP architecture can make this boundary executable. Retrieval produces a slice with a coverage status. Formalisation produces assertions with source spans and assumptions. The solver produces a verdict over exactly those assertions. A promotion circuit decides whether that verdict answers the original question. These are separate responsibilities, and the last one cannot be omitted merely because the solver is exact. The scope-aware result type is a candidate design contribution motivated by this audit, not a feature already verified throughout the supplied systems.

HoloProof remains one of the more promising architectural experiments because it already separates intuition from formal checking. The new finding does not invalidate that separation; it strengthens it. Intuition must not only be prevented from fabricating a proof. It must also be prevented from making an incomplete set of premises look like the whole world. That is a more demanding and more useful standard for trustworthy agentic reasoning.

## 10. Maintaining alternative hypotheses and selecting questions

autoResearchLib tests a bounded but important aspect of scientific and agentic reasoning: maintaining several candidate explanations and choosing questions that distinguish them. The experiments show that reducing uncertainty is not equivalent to preserving recoverability. Under noisy answers, a question can make the frontier more concentrated while eliminating the correct explanation. This motivates reversible, provenance-aware belief updates rather than confidence-only state compression.

### 10.1 Representing a frontier of alternative explanations

autoResearchLib investigates a problem that a single best answer cannot represent well: several explanations may remain compatible with incomplete evidence. Its system maintains a frontier of theories, updates them as observations arrive, and can choose questions that discriminate among them. This is relevant to a language model because ambiguity and partial information are normal, not exceptional. A model that commits too early may be unable to recover when a later sentence changes the interpretation. [R13]

The implemented experiment is bounded. Its domains, observation mechanisms, and theory families are substantially designed in advance. It does not demonstrate open-ended scientific theory invention. The value lies in making alternatives, evidence, and collapse visible. The system can be evaluated not only by whether its first-ranked answer is correct, but also by whether the correct explanation remains available for later recovery.

The audit reran all seven repository experiments. The progression begins with eighteen controlled cases and expands through questioning, masking, structural ablations, a larger semi-synthetic benchmark, novelty conditions, and multi-step recovery. The final experiment produced 2,448 traces across policies, question budgets, and answer conditions. Those traces are more informative than a single headline score because they reveal how a policy behaves as evidence changes. [A03]

The early results show a real benefit from richer observation. At one intermediate prefix in the first experiment, a richer observer achieved about 83.3 percent classification accuracy compared with 33.3 percent for the coarse observer. By the final prefix both reached full accuracy. This does not mean the richer observer discovered the domain ontology from nothing. It means that exposing more relevant distinctions earlier changed which theory could be selected. The lesson is representational: a frontier cannot use information that the observer discards.

The expanded benchmark contains 102 cases over seven workflow families, divided into development and test sets of 51 cases each. Four prefixes per test case yield 204 test traces. On the first prefix, frontier-top accuracy was about 23.5 percent, below the naive Bayes baseline's 41.2 percent. At the second prefix, the frontier reached about 94.1 percent and retained the correct theory in every test case; later prefixes reached full top accuracy in this controlled benchmark. This is a trajectory of evidence use, not uniform dominance over a baseline. [R13; A03]

The distinction between top accuracy and truth retention is genuinely useful. A system may rank the wrong explanation first while keeping the right one available. A later question can then rescue it. But retention alone can be trivial: a system that keeps every theory retains the correct one whenever it was present



initially. A persuasive evaluation therefore needs a retain-all control and a measure of frontier cost, such as the number of alternatives, verification work, and decision latency. The useful frontier is not merely inclusive; it is economically recoverable.

The structural ablations also restrain attribution. Several disabled mechanisms produced the same aggregate accuracy as the baseline in the tested conditions. That does not prove the components are useless, but it prevents crediting them for an improvement the ablation did not observe. The field often treats architectural complexity as explanatory evidence. Here the ablation table permits a more disciplined statement: some machinery is implemented and plausible, but its contribution remains unmeasured on these cases.

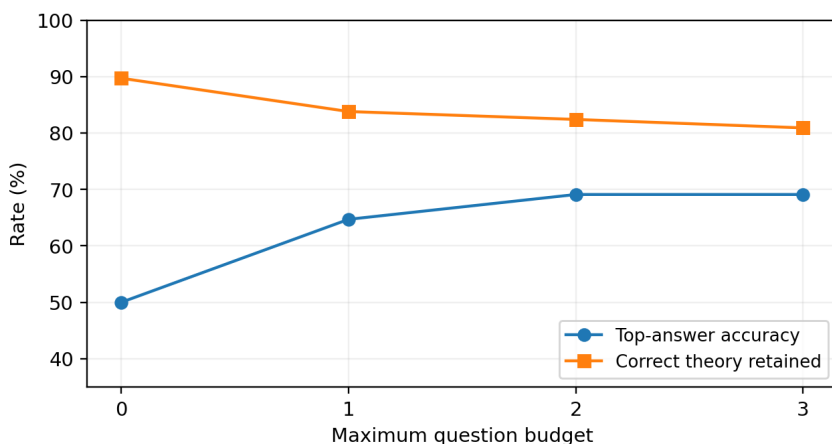
The core idea worth carrying forward is a first-class representation of unresolved alternatives. In a SOP-based system, a parse, a causal explanation, or a plan could be a collection of candidate circuits rather than one irreversible commitment. Evidence would prune or rerank them. The challenge is to retain recovery without allowing unbounded expansion. This connects frontier management with the earlier exact representation's combinatorial growth problem.

## 10.2 Observed behaviour under clean and noisy answers

A natural policy asks the question expected to reduce uncertainty most. In a closed and reliable model, that can be sensible. The repository compares an information-gain policy with random questioning and a cheaper symbolic top-domain heuristic. It also distinguishes clean answers from masked evidence with noisy answers and from deliberately adverse answers. This is exactly the kind of separation a trustworthy agent needs. [R13]

In the clean condition of the multi-step experiment, information-gain accuracy began at approximately 97.1 percent and reached 100 percent with a budget of two questions. The top-domain heuristic matched that accuracy trajectory in the reported clean results. Thus the experiment supports the usefulness of additional information but does not establish that the more elaborate information-gain computation is necessary there. An efficiency claim would need to count the cost of selecting questions as well as the number asked. [A03]

The noisy-answer condition is more revealing. With masked evidence and no questions, information-gain top accuracy was 50.0 percent and truth retention about 89.7 percent. At a budget of two questions, accuracy rose to about 69.1 percent, but truth retention fell to about 82.4 percent. At three questions, accuracy remained about 69.1 percent while retention fell further to about 80.9 percent. The policy improved immediate decisions for many cases while making some initially recoverable cases unrecoverable. [A03]



**Figure 4. Better immediate answers can coexist with less recoverability.** Information-gain policy under masked evidence and noisy answers. These rates come from the repository's bounded semi-synthetic protocol, not a deployed agent. Source: A03, autoResearchLib experiment 7.

Entropy decreased over the same sequence, from about 1.107 without questions to 0.427 at budget three. A narrower distribution therefore did not mean that the remaining alternatives were more faithful to the hidden truth. The system was learning from answers under assumptions that were not always satisfied. Some uncertainty reduction was useful; some was overcommitment to misleading evidence.

The deliberately adverse condition makes the effect unmistakable. The generator chooses answers designed to damage correctness and truth retention. After one information-gain question, top accuracy fell from 50.0 percent to about 35.3 percent, and truth retention from about 89.7 percent to about 35.3 percent. Entropy nevertheless fell from about 1.107 to 0.320. The policy became substantially more concentrated while losing much of its access to the correct explanation. [A03]

This is not evidence that information gain is mathematically wrong. It is evidence that an acquisition objective depends on the model of the answer channel and the update rule. If the system assumes answers are reliable when they are not, a highly discriminating question can become a highly destructive filter. The very question that most efficiently separates alternatives can eliminate the truth most efficiently when answered incorrectly.

The archived analysis already recognises this failure as harmful contraction. The contribution of this edition is not to invent that interpretation, but to rerun it, place it beside the other repositories, and identify its architectural consequence. The same pattern appears in the HoloProof scope problem and the metric similarity problem: a component produces a locally meaningful signal, and another stage gives it more authority than its assumptions justify.

This is one of the strongest insights for agentic systems in the collection. An agent should not be optimised only to become less

uncertain or to act sooner. It should preserve a path back from misleading evidence. In practical terms, evidence needs source reliability, alternatives need reversible status, and destructive pruning needs stronger justification than ordinary reranking. Those requirements are not supplied by a holographic representation alone, but they can be expressed as explicit circuit policies.

### **10.3 Research significance: information gain with recoverability**

The next version of the frontier system should treat an answer as an observation with a provenance and error model, not as a guaranteed branch selector. A question asked of a trusted database, a human witness, an external language model, or an adversarial document creates different evidence. The system should know which source supplied the answer and whether later checks can contradict it. This is an architectural requirement derived from the rerun failure mode, not a claim that the current library already implements a complete trust model.

One concrete proposal is reversible pruning. An explanation could become inactive without being deleted, retaining the answer and rule that caused its demotion. If later evidence contradicts that answer, the explanation can re-enter the frontier. This requires storage and update logic, so it should be evaluated against its cost. It also avoids the trivial retain-all solution by allowing inactive candidates to occupy a cheaper representation than actively evaluated ones.

Another proposal is risk-sensitive question selection. Expected uncertainty reduction should be combined with the cost of a wrong answer and the probability of obtaining reliable evidence. A question that sharply separates all theories but relies on an

unreliable source may be worse than a modest question whose answer can be independently checked. The experiment should measure final task success and recoverability, not only entropy or immediate classification.

The novelty experiments in `autoResearchLib` add a related caution. The reported detector flagged only a portion of open and hybrid cases. In the rerun summaries, novelty flags appeared in about 53.3 percent of open cases and 41.7 percent of hybrid cases, compared with about 5.2 percent of in-domain traces. A reported absence of a certain false-closure event under the policy's definition does not imply complete novelty detection. The system can avoid one dangerous form of commitment while still missing many unfamiliar cases. [R13; A03]

This matters for the programme's ambition to create new semantic circuits. A frontier containing only known theory families may not include the right explanation at all. Preserving alternatives is insufficient when the candidate language is deficient. The system needs a separate operation for proposing an extension, together with tests that distinguish a genuine new explanatory structure from a relabelled old one. `Eos Gate` offers an intervention-based validation pattern; `autoResearchLib` offers an alternative-management pattern. Their combination is a plausible research direction, not a completed result.

A useful experiment would introduce a small number of genuinely out-of-family processes, provide noisy observations, and allow an agent to propose a new executable theory. Acceptance would require better held-out prediction, a trace of the new assumptions, and successful interventions where the world permits them. A simpler baseline could add an explicit “unknown family” without generating a new theory. The proposed system should beat that

baseline in useful recovery rather than merely producing more elaborate descriptions.

For explainability, the frontier should show not only the winning answer but also why alternatives were retained or removed. For trustworthiness, it should reveal which uncertain source caused an irreversible decision. For efficiency, it should avoid spending equal work on every possibility. These three goals are in tension, and the library's experiments make that tension measurable.

The strongest lesson is therefore not “ask more questions.” It is that questions are computational interventions on a belief state. They can improve or damage that state depending on the answer channel and update policy. A circuit architecture can expose that intervention explicitly: select a question, obtain an answer, validate its source, update alternatives, and record a reversible justification. That is a concrete way the VSA-era experiments can contribute to more accountable agentic systems even when VSA is no longer the central representation.

# 11. VSA-based routing of neural computation

vsaLLM tests a narrower role for VSA/HDC: request-level routing in a sparse mixture-of-experts language model. The repository is deliberately conservative about claims and defines matched controls and evidence gates. In the supplied snapshot the protocol is substantially implemented, but a complete training-to-deployment result establishing superiority was not produced. The experiment remains interesting because it asks whether VSA can be useful as a low-cost control signal even if it is not the semantic substrate of the model.

## 11.1 VSA-PathMoE: VSA as a routing signal

The vsaLLM repository takes a strategically different position from the early algebraic reasoners. It retains a neural language model and uses a VSA router to select a sparse path through small tensor experts. Attention and a shared feed-forward path remain active. VSA is therefore not asked to perform all linguistic inference. It is asked to choose which learned computation should run for a request. [R15]

This is an important narrowing of the hypothesis. A request often has reusable structure: a topic, a task type, a combination of constraints, or a history-dependent state. A compositional representation might map that structure to a useful route more predictably than an opaque learned router or a simple hash. But the claim is about conditional computation. The quality of the neural experts and the shared model remains part of the explanation for any successful output.

The default configuration has six decoder layers, width 192, and 64 request-stable routes. It contains 10,029,504 total parameters and an arithmetic active count of 1,755,840 per request. Those figures describe the implemented model and selected tensor path. They do not measure resident process memory, memory bandwidth, wall-clock latency, energy, or trained language quality. The repository itself makes this distinction explicitly. [R15]

Request stability is a substantive design choice. A route that changes unpredictably during generation can complicate caching and make performance difficult to interpret. A stable route can reduce switching, but it may also be too rigid for a long request that changes topic or enters a different reasoning phase. The experiment should therefore test both the benefit of stability and the cost of committing too early. A route is another kind of hypothesis about what computation will be useful.

Causality is equally important. A router evaluated on a continuation must not use information from the future continuation when deciding the route. Otherwise the benchmark leaks the answer into the computation selector. The repository's protocol treats the raw continuation boundary and the causal routing conditions as explicit parts of the experiment. This is good design discipline because routing improvements can otherwise be confused with access to additional information.

A fixed or inexpensive router is not novel by itself. Hash Layers already investigated non-learned routing for sparse models. The relevant comparison is therefore not only with a dense model, but also with routing policies that preserve similar computational constraints. A VSA route should justify its additional representation machinery through better quality, compositional transfer, stability, or some other measured property. [L09]



The audit passed 28 Node tests and 52 Python tests associated with the repository. These tests support its protocol machinery and implemented invariants. No complete training-and-deployment experiment was performed here, and the supplied project's own status is “protocol implemented; result pending.” There is no basis in this edition for claiming that VSA-PathMoE is faster, smaller in process memory, or better at language than its controls. [A02, A03]

Nevertheless, the repository belongs in the innovation account because it turns a broad ambition into a falsifiable, comparatively bounded experiment. It asks whether VSA is useful at a particular interface in a language model. That is a more credible test than requiring a single algebra to replace token prediction, memory, reasoning, and language generation at once.

## 11.2 Matched controls and evidence gates

The `vsaLLM` protocol distinguishes several baseline roles. A published `TinyStories` model is an external reference, a larger published model is a capacity anchor, and two locally trained dense controls match the sparse model's active and total parameter budgets. Those controls answer different questions. A small dense model helps test whether sparse selection is better than simply using a smaller model. A total-size-matched dense model helps test whether any quality difference reflects total capacity rather than routing. [R15]

The quality metric uses conditional bits per UTF-8 byte at the same raw continuation boundary. This is meant to make comparisons less dependent on different tokenisations and prompt slicing. It does not make every difference in training data or model design disappear. That is why local matched controls remain necessary. An external published checkpoint is informative but cannot substitute for a carefully matched training comparison.

The official workflow also separates training on a DGX Spark environment from deployment measurements on commodity x86 hardware. A manifest freezes the relevant artifacts and hashes before transfer. The deployment stage checks the handoff and runs defined thread-count conditions under a recorded physical-core affinity. Reports require the appropriate control matrices and reject missing or inconsistent lineage. These are implemented research constraints, not evidence that the complete official run already happened. [R15]

This protocol is unusually relevant to the programme's practical goal. A model can look efficient when counted by active arithmetic parameters and still perform poorly on a CPU because of tensor shapes, cache behaviour, memory movement, and dispatch overhead. A sparse design also carries inactive weights that may remain resident. Training in one environment and benchmarking in another creates additional opportunities for accidental substitutions. A frozen artifact boundary makes such mistakes visible.

The protocol's insistence on a complete set of controls also counteracts a common failure mode in AI-generated reports: choosing the comparator that makes the new method look best. A method can beat an external checkpoint on one prompt set while losing to a locally trained control, or appear fast under one thread policy while using more favourable core placement. The report should retain those differences, not select one convenient number as the conclusion.

A further useful test would examine composition in the router itself. Train on requests combining some task and content factors, then withhold particular combinations. Does the VSA route select experts that remain useful for the unseen combination, or does it behave like another opaque code lookup? Compare with hashing,

random routes, a learned router, and an exact symbolic dispatch policy when the factors are known. That experiment would test the proposed compositional advantage rather than merely the existence of sparse execution.

Another test should separate routing from expert quality. Hold the learned expert weights fixed and vary only the route. Then hold the routing policy fixed while varying expert organisation. A large improvement in the second experiment would not establish that VSA routing was responsible. A large improvement in the first would focus attention on the selector. These crossed interventions are the same causal discipline that made the Eos decoder diagnosis useful.

The innovative possibility is a route representation that supports interpretable, compositional, and stable selection without expensive per-token routing. The unproven part is whether that possibility survives matched quality and full-system efficiency comparisons. The book should preserve the protocol as a promising research instrument while refusing to turn its safeguards and parameter counts into a result they have not yet produced.

### **11.3 Current evidence boundary and required next result**

The user's concern is that the explored holographic representation and reasoning equations have not been sufficiently powerful and general. The evidence supports that concern as a judgment about this programme's current implementations. It does not support the stronger statement that VSA cannot express general computation. Published constructions of vector-symbolic Lisp, for example, explicitly include cleanup memory and describe representations of the elementary operations needed for general computation. Such constructions change the resources and operations under consideration. [L07]

This is not a reason to dismiss the practical concern. A construction capable of representing a general interpreter does not show that it learns useful language behaviour cheaply, handles ambiguity well, or outperforms an ordinary implementation of the same interpreter. Expressivity and efficiency are different properties. If a VSA system obtains generality by reintroducing large exact cleanup tables and external control, the programme must measure the whole construction rather than celebrate the vector notation alone.

Other work shows that the operation family can be expanded. Resonator networks address factorisation of distributed structures, and residue-number representations support forms of arithmetic within high-dimensional representations. These are relevant precedents when proposing “new operations.” They do not establish that arbitrary natural-language reasoning reduces to one operation, nor that these particular repositories have implemented those mechanisms. They demonstrate that the current code’s operator set is not the only conceivable VSA operator set. [L04, L08]

The right conclusion is therefore conditional. The tested simple algebras do not justify a universal reasoning interpretation. Some measured failures are caused by code or decoding and can be repaired. Some arise from representational invariances that remove required distinctions. Some arise from resource growth or interference. General computation may be representable with additional machinery, but the practical value of that machinery remains an empirical question.

This suggests a better search criterion for extensions. A new operator should identify a task distinction that current operators cannot preserve or transform correctly. It should state its input and output types, its exact or probabilistic laws, its resource costs, and its failure modes. It should then be compared with an explicit symbolic implementation of the same transformation. Without that

comparison, a new holographic operation may merely be a more expensive encoding of a conventional algorithm.

For example, an operator for perspective change should preserve the difference between a fact and a character's belief. An operator for counterfactual repair should expose which variables are allowed to change and verify that the proposed intervention satisfies the target condition. An operator for proof composition should preserve assumptions and source scope. These are semantic requirements first. A vector implementation is valuable when it satisfies them efficiently, not when its output is merely similar to an expected vector on a few examples.

The emerging architectural boundary is productive: VSA remains available as a carrier and selector, while circuits organise the operations and their obligations. A future discovery could move that boundary. If a new representation supports a difficult transformation more cheaply and reliably, the circuit implementation can adopt it. The programme need not make an irreversible philosophical choice between vectors and symbols. It needs a framework in which a representation earns responsibility through evidence.

VSA-PathMoE illustrates one such responsibility already well enough defined to test. It may or may not succeed after training. Either outcome would be informative because the claim is bounded. That is the direction in which the broader programme should move: away from asking one family of operations to justify an entire language model, and toward experiments in which a specific component can clearly deserve or lose its place.

## 12. SOP Lang circuits as the current central architecture

The combined evidence motivates the current move toward SOP Lang circuits. This is not presented as proof that symbolic circuits are universally superior. It is an engineering response to repeated requirements observed across the experiments: preserve role identity, expose premises, represent operations explicitly, support local correction, and separate approximate retrieval from authoritative execution. VSA/HDC can remain inside such an architecture as an index, associative memory, or router without carrying every semantic obligation.

### 12.1 Make semantic structure executable

The proposed move to SOP Lang circuits should not be understood as replacing one metaphor with another. The crucial decision is what becomes the canonical object of knowledge. If the canonical object is a fixed approximate vector, then exact identities, roles, assumptions, and derivations must be recovered from it or stored elsewhere. If the canonical object is an executable semantic circuit, those distinctions can be explicit, while vectors remain derived views used for retrieval or selection.

This is a proposal informed by the audit, not a claim that a complete SOP implementation was found in all fifteen repositories. The collection contains virtual machines, constrained languages, symbolic representations, event systems, and formal-verification components. The human researcher identifies SOP Lang as the next central architectural metaphor. This chapter specifies what that transition should preserve from the experiments and what would have to be demonstrated to justify it.

An executable semantic circuit is more than a graph drawing. Its nodes perform defined operations. Its edges carry typed values or references. Its execution has an order determined by dependencies and any explicit control semantics. Its result records the assumptions and sources under which it is valid. A graph containing the words “reason,” “understand,” and “summarise” is not yet such a circuit; each operation must resolve to executable behaviour or to a clearly marked external proposal step.

A small example connects several repositories. A document states that an artifact was moved, then a later report corrects its destination. An ingestion circuit produces candidate event records with source spans. A validation circuit admits one interpretation into a versioned world. A state-transition circuit updates the current location while retaining the earlier event. A retrieval circuit finds relevant events for a question. A summarisation circuit selects accepted records and produces a concise answer. A response checker verifies that each asserted location is licensed by the selected world version.

None of those operations requires a universal vector equation. Some can use exact indexes; some may benefit from VSA association; some may call a language model. Their responsibilities differ. The external language model may propose a translation or a phrase, but it should not silently decide whether a source record was superseded. The approximate retriever may suggest an event, but it should not erase the event's timestamp. The circuit structure makes these boundaries explicit.

The exact and elastic representation experiments provide the memory insight behind this design. A summary is not the entire state needed for revision. A factorised expression can avoid eager expansion while retaining the ability to compute alternatives later. A growing atom universe requires identity management. An event

log supports reconstruction. These are not incidental implementation details around intelligence; they are part of the knowledge object's semantics.

Prior work on equality saturation and provenance provides important boundaries for novelty. E-graphs already represent and optimise equivalence-rich computations, and provenance semirings provide algebraic accounts of how results depend on inputs. A future SOP architecture should use such ideas where appropriate rather than relabel them as inventions. The research opportunity is in their integration with learned or proposed semantic circuits, uncertainty management, and task-sensitive representation choice. [L10, L11]

The canonical circuit should therefore support more than execution. It should support inspection, invalidation, substitution, and controlled promotion from a task-local object to long-term knowledge. These operations address the weaknesses repeatedly observed in the repositories: hidden symbolic assistance, ambiguous result scope, irreversible frontier pruning, and summaries that lose the state required for correction. A successful implementation would not prove that circuits solve intelligence. It would give the programme a more reliable unit on which to accumulate evidence.

## 12.2 Operator obligations derived from the experiments

The first obligation is semantic preservation across representation changes. A circuit that converts an event record to a vector must state which distinctions it preserves. The VSAText probe shows that an unordered word bundle does not preserve actor and recipient. The spock probe shows that cosine does not preserve positive magnitude-based confidence. The metric backend shows that XOR does not preserve its numerical distance. These are not reasons to



prohibit the operations; they are reasons to type their outputs and restrict the transformations that follow.

A practical implementation could attach capabilities to each representation. An exact role record might support identity and argument-position queries. A topic vector might support approximate topical retrieval but not role reconstruction. An unquantised linear feature encoding with a registered dual readout might support exact evaluation of specified linear properties within numerical tolerance. The planner would select an operation only when the representation's contract provides the required capability. This capability-aware dispatch is a proposed integration mechanism, not an implemented result of the present audit.

The second obligation is scope preservation. HoloProof demonstrates that a correct local solver result can be misleading when detached from its premises. A circuit result should therefore carry its world, accepted assumptions, checked slice, and coverage status. Composition must preserve those fields. A downstream summariser cannot turn “not established from this slice” into “false in the knowledge base” without another operation that justifies the stronger claim.

Scope also matters for negation and hypothetical worlds. An assertion may be false in a scenario, unknown in the accepted world, and believed true by a character. These are different values, not different similarity levels of one undifferentiated proposition. A circuit can represent the world or perspective as an explicit input. This makes the operation inspectable and gives tests a precise target. It also allows approximate retrieval to be restricted to the relevant context rather than hoping that one global vector separates all perspectives.

The third obligation is update reversibility and provenance. autoResearchLib shows that a question can shrink the frontier

while eliminating the truth. A safe update should record what evidence caused a candidate to be demoted and whether that evidence can later be withdrawn. VSABrains2's event-oriented machinery provides a related positive pattern: retain transitions so that a state can be reconstructed. The proposed circuit system should make corrections replayable rather than overwriting an opaque memory and losing the reason for its current state.

Reversibility here does not mean every operation has an algebraic inverse. A lossy summary may be irreversible by itself. The system can still support correction by retaining its source circuit and recomputing the summary. This is a crucial distinction. The programme need not discover a magical inverse for every compressed representation if it can manage source-backed recomputation efficiently. The exact carrier's failed multiplicity recovery and the elastic carrier's retained sums illustrate why these two notions of reversibility should not be confused.

The fourth obligation is resource-bounded growth. An exact representation can expand combinatorially; a noisy memory can overload; a frontier can keep too many alternatives; a neural router can activate more computation than expected. The circuit should expose the resource consequence of an operation before or during execution. A planner can then choose lazy factorisation, a bounded candidate set, a new memory chunk, or an explicit incomplete result.

These four obligations form a concrete research agenda because each has a failing example already present in the collection. They are not abstract aspirations such as “be explainable” or “be trustworthy.” They can be tested through metamorphic cases, counterexamples, and controlled resource sweeps. A representation change should preserve a role-sensitive answer when it claims role preservation. A scoped result should not

strengthen itself after a retrieval permutation. A correction should restore alternatives affected by withdrawn evidence. A growth policy should improve useful task performance under a declared total budget.

The potential innovation lies in making these obligations executable and compositional. Individual type systems, proof objects, provenance records, and resource limits are not new. Their integration around an extensible language of semantic circuits may be useful if it enables a system to grow knowledge, discover reusable operations, and remain correctable without relying on one opaque global representation. The next implementation must demonstrate that usefulness rather than assume it from the architecture diagram.

### **12.3 Circuit induction without a hidden semantic engine**

The programme permits human researchers and coding agents to construct new executable knowledge from documents. That is a practical way to begin. A language model can read a manual, propose entities and rules, and generate SOP declarations or implementations. But the scientific claim must then be about the resulting executable system and its validation, not about an autonomous symbolic model having learned everything without a language-model teacher.

The Eos experiment gives a useful template. The learner produces a candidate law, and an independent world checks predictions and repairs. In a circuit-learning experiment, an agent would propose a reusable transformation, while held-out inputs and independent validators determine whether it deserves acceptance. The proposal mechanism can be powerful and opaque as long as the accepted artifact's behaviour is tested under conditions that do not reveal the answer to the proposer.

A first controlled task could involve event descriptions with varying names, passive constructions, and temporal order. The agent would propose circuits that map the descriptions to canonical events and answer state queries. Training examples would contain some combinations of roles and events; test examples would withhold combinations and introduce corrections. A literal template baseline, a conventional parser-plus-database baseline, and the proposed circuit learner would receive the same information. The evaluation would count semantic errors, unsupported answers, repair cost, and execution work.

A second task could extend Eos Gate from additive laws to small conditional programs. The candidate language might allow comparisons, conjunctions, bounded counting, and piecewise arithmetic. The generator would include both representable and out-of-family laws. A useful learner would construct a circuit when evidence supports one, preserve alternatives when underdetermined, and report a language limitation rather than inventing confidence. VSA could retrieve candidate fragments or similar past laws, but exact execution would determine the candidate's predictions.

A third task could focus on summarisation from a versioned knowledge base. The challenge would not be to produce the most fluent paragraph. It would be to select the facts required by a query, omit superseded claims, preserve uncertainty, and attach a source-backed explanation for every accepted statement. A language model could realise the selected content in prose. A separate checker would test whether the prose adds unsupported commitments. This would make a useful contribution to trustworthy AI even before open-domain language quality becomes competitive.

The danger is a hidden reasoner in the host language. If the kernel contains domain-specific rules that solve every benchmark while the SOP files merely trigger them, the system has not demonstrated learning of executable knowledge. The audit's hybrid attribution problem would recur under a different name. Kernel primitives should therefore remain small, explicit, and general enough that the benchmark's domain knowledge is visible in the generated circuits or data structures.

Conversely, requiring the kernel to contain only a tiny fixed algebra can make the circuit language impractical. The lesson is not to ban new primitives, but to expose their semantics and cost. A specialised solver or parser can be a valid circuit component when its contribution is acknowledged. The system should distinguish a primitive supplied by an engineer, a circuit proposed by an AI, and a structure induced automatically from data. These origins affect claims about learning, not whether the component may be useful.

The research programme becomes more credible when progress is measured as reusable capability rather than architectural purity. A newly accepted circuit should enable a previously unsupported operation, reduce the cost of an existing task, improve correction behaviour, or generalise under a declared transformation. If it does none of these, adding it merely increases the system's vocabulary. That standard gives the human and the AI agents a shared target: not more mechanisms, but more justified and reusable computation.

## 13. Research conclusions and open experiments

The final assessment is intentionally provisional. The experiments contain several potentially publishable mechanisms and several informative failures, but they do not yet justify a general VSA/HDC-based alternative to an LLM. The purpose of this chapter is to state what appears worth continuing, which claims remain unsupported, and which experiments could reverse the present conclusion. The research programme remains open to new VSA operators or representation families if they demonstrate capabilities not obtained by the current variants.

### 13.1 Candidate contributions supported by the current evidence

The strongest candidate contribution in the memory experiments is interference-aware placement. VSANash contains a concrete policy, competing strategies, and a rerun improvement in a specified operating region. The next question is whether that region remains useful after counting build cost, updates, memory, and exact verification. A successful continuation could produce a valuable systems result without claiming a new universal theory of memory. The high-load and two-hop collapses are part of the contribution's boundary, not inconvenient details to remove.

The strongest contribution in Eos Gate is an executable account of a discovered property that supports transfer and verified intervention. The semantic null-space learner is restricted, but the protocol connects observations, explanation, prediction, and repair. The new decoder audit adds a distinct result: the observed HDC

error was removable in the tested full-rank linear setting. Neither ordinary null-space computation nor dual readout should be claimed as new mathematics. Their disciplined combination in this experimental setting is what can be developed further.

The representation backends contribute a map of growth choices. Sparse sets reveal the importance of algebraic semantics; elastic chunks preserve update state; exact terms replace collision noise with explicit combinatorial cost. The candidate innovation is a policy that chooses growth and representation according to the operation required. That policy is not yet demonstrated. It becomes a credible next experiment because the existing implementations already expose the relevant failure modes and resource trade-offs.

The frontier experiments contribute a recoverability perspective. A system should be judged not only by the answer it ranks first, but also by whether later evidence can rescue it. The rerun noisy and adverse conditions show why entropy reduction is not enough. A source-aware, reversible frontier update mechanism would be a meaningful extension, especially when combined with executable candidate theories. Again, the proposed extension must be distinguished from the bounded library that motivated it.

HoloProof contributes a promising separation of proposal and verification, together with a newly demonstrated boundary error at the retrieval-to-solver interface. The finite Boolean probe shows why proof authority requires scope authority. A coverage-aware result contract could become a practical contribution to retrieval-augmented reasoning and agentic systems more broadly. The result is not specific to hypervectors, which increases rather than decreases its usefulness.

VSA-PathMoE contributes a carefully bounded experiment about selecting computation. Its strongest current asset is an executable protocol that resists misleading parameter and baseline

comparisons. The candidate mechanism remains unvalidated by a complete training result in this edition. It should remain in the programme if the cost of finishing the controlled experiment is justified, not because its name contains both VSA and language model.

The prediction and narrative repositories contribute reusable architectural fragments rather than a demonstrated alternative LLM. BSP's explicit copy and compression operations, VSAVM's execution machinery, SomaVM's bounded local-learning substrate, VSABrains2's episodic structures, and VSAText's narrative schemas all contain ideas that can be separated from their weaker claims. The invalid negation score, weak checkpoint result, role-erasing encoder, and proxy drift metric tell the programme which capabilities still need real tests.

Taken together, the collection supports a shift in emphasis. The most durable advances are not a single holographic reasoning equation. They are mechanisms for making representation, execution, correction, and evidence explicit. Some remain naturally vector-symbolic; others are already symbolic programs or structured memories. A useful book should not force them back into a homogeneous VSA story merely to preserve the original label.

### **13.2 Experiments that could reverse the current conclusion**

The present judgment should be reversible. To return VSA/HDC to the centre of the architecture, a new result would need to demonstrate more than better accuracy on a familiar synthetic task. It should show that a representation and operator family supports a substantial class of useful transformations with a favourable total cost and clear failure behaviour. The comparison



should include an explicit symbolic implementation of the same operations, not only a weak neural or lexical baseline.

A decisive representation experiment would hold semantic information fixed and vary the carrier. It would include exact records, factorised circuits, dense VSA, elastic VSA, and a simple identifier or hash control where meaningful. Tasks would demand identity, role sensitivity, temporal updates, composition, and correction. Measurements would include successful operations, rejected unsupported operations, storage growth, and the work required for verification. This would distinguish representational power from a wrapper that quietly does all the reasoning.

A decisive operator-discovery experiment would begin from a documented failure such as non-distributive binding or missing scope. The agent would propose an operation together with laws it claims to satisfy. Automatically generated counterexamples would test those laws before downstream tasks were run. A successful operator would then need to improve a task not used to design it. This sequence would prevent a plausible algebraic description from becoming a foundation before its basic identities are checked.

A decisive learning experiment would expose the system to new combinations and new operations, not only new surface strings. The accepted knowledge would be inspectable circuits or explicit structures, and the test would measure whether those objects support transfer. The learning agent should not receive hidden generator parameters, and test generation should include cases outside the proposed language. Failure to represent a law should remain distinguishable from failure to infer one that is representable.

A decisive memory experiment would include corrections and deletions. A system can look excellent when knowledge only accumulates, yet become unreliable when an old claim is

withdrawn. The experiment would verify that accepted answers stop depending on deleted evidence, that summaries are recomputed where necessary, and that dormant alternatives can be recovered. This would directly connect the programme with trustworthy AI rather than treat trust as a separate layer of documentation.

A decisive agent experiment would combine approximate retrieval, exact checking, and question selection under misleading evidence. It would measure task completion, unsupported claims, harmful actions, and recovery after corrections. Silence would not count as universal success, as the BSP probe warns. Low entropy would not count as justified confidence, as `autoResearchLib` warns. A solver verdict over a slice would not count as a full-world answer, as `HoloProof` warns. The evaluation would be stronger because its failure modes are already known.

Finally, a decisive efficiency experiment would measure whole-system cost on the intended CPU deployment. It would include index construction, model state, source witnesses, cleanup dictionaries, candidate checking, and update work. Theoretical active arithmetic, byte width of one atom, and one fast query stage are insufficient. The `vsaLLM` protocol offers useful discipline here even if its particular routing model ultimately loses to a dense control.

These experiments are not proposed as a grand benchmark that must be built before any further work can happen. Each can be run as a small falsification study. The programme should choose the next one according to which uncertainty most affects an architectural decision. That is a practical meaning of efficient research: spend computation on a result that can change what is built, rather than on another large table whose interpretation remains ambiguous.

### 13.3 Human direction, AI implementation, and AI audit

The human contribution documented in this conversation is strategic and methodological. The researcher chose the ambition of a language-capable system, encouraged the exploration of alternative representations, and now asks whether the evidence justifies leaving VSA/HDC as the centre. The explicit feedback also preserves uncertainty: poor results may come from engineering or from an incomplete operator set. That combination of direction and willingness to revise is important. It prevents both premature abandonment and indefinite loyalty to an attractive metaphor.

The repository artifacts show AI assistance at several levels. Eos Gate discloses drafting, refactoring, documentation, and article assembly with coding tools, while explicitly rejecting a claim of autonomous scientific discovery. VSANash contains an Antigravity-attributed review proposing more informative retrieval diagnostics. VSAText contains Codex-attributed criticism of a missing pattern runtime. These records justify attributing those particular review observations or disclosed assistance to AI. They do not justify assigning every design choice to an identifiable agent.

This edition adds another layer: an AI inspected AI-assisted experiments and wrote executable tests of its own interpretations. The confidence-scaling probe, operator counterexamples, elastic payload measurements, exact-growth probe, dual decoder, empty-answer negation test, scope-sensitive Boolean checks, and VSAVM repair are the clearest new artifacts of this audit. They are not independent replications by a separate institution. They are inspectable interventions whose evidence can be reproduced without trusting the literary account.

The circularity risk is real. An AI can produce a plausible implementation, another AI can praise the documentation, and a third can write a convincing book without anyone checking the

mechanism. This edition tries to break that circle through direct source references and small independent tests. The approach is still fallible. The finite Boolean adapter covers only its declared fragment. The decoder experiment uses one synthetic data world. The selected tests do not cover every file. Exposing these limits is part of making another audit possible.

Research automation itself has prior art. Systems such as The AI Scientist investigate automated pipelines for proposing, executing, and reporting experiments. The distinctive question here is not whether an AI can write a research document. It is whether an AI can improve the explanatory quality of an existing research programme by separating implementation, measurement, and interpretation. That contribution should be judged through the audit artifacts and the decisions they support, not through the author's fluency. [L12]

Several observations in this book are deliberately unglamorous: a missing module entry point, an empty development suite, a declaration-order exception, a metric with the wrong name, a scorer that accepts silence. These are not distractions from the search for intelligence. They are part of the environment in which claims about intelligence are produced. Removing them can reveal a valid mechanism, as in Eos decoding, or remove false support, as in BSP negation. Both outcomes improve the programme.

My final assessment is that the human's architectural pivot is justified by the available evidence, but should remain open to revision. No examined result establishes a general-purpose language model founded on a universal holographic reasoning algebra. Several results support narrower uses of VSA, and one important apparent weakness was repaired without changing the representation. The strongest cross-repository direction is to make semantic structure and executable obligations explicit, allowing

approximate representations to contribute where their value is measured.

The next architecture should outgrow its metaphors: retain VSA where it improves a task, prefer an exact index where it does not, admit circuits that survive new tests, and revise beliefs when evidence changes. Explainability requires inspectable operations; trustworthiness requires scoped authority and correction; efficiency requires whole-system measurement. These remain research objectives, not achievements already established by the repositories.

# Appendix A. Repository and execution evidence

The identifiers below refer to the uploaded snapshots, not to an inferred Git commit. Full archive SHA-256 values are in the companion manifest. Paths identify the main evidence used for each discussion. The machine-readable source register also hashes the individual files, allowing a later audit to distinguish a changed implementation from a changed interpretation.

## R01 • spock

A small numeric kernel and its semantic interpretations. Positive scalar confidence disappears under the inspected cosine-based comparison. The selected repository runner passed 350 checks.

Principal source paths: `src/kernel/primitiveOps.js`; `src/kernel/vectorSpace.js`; `docs/THEORY.md`; `docs/WHY_IT_WORKS.md`.

## R02 • HDC-RE

Hybrid proof and query paths combine similarity with direct matches, chains, and rule machinery. The selected suite passed 476 tests and failed thirteen. The failures are retained rather than converted into a general conclusion about the algebra.

Principal source paths: `src/reasoning/prove.mjs`; `src/reasoning/query.mjs`.

## R03 • VSARepresentations

The main source for alternative carriers, operator counterexamples, elastic allocation, and exact combinatorial growth. The selected unit suite passed 807 of 808 tests; the failed CommonJS entry-point test is a packaging issue. A separate twelve-configuration evaluation is recorded in A03.

Principal source paths: `src/hdc/strategies/sparse-polynomial.mjs`; `src/hdc/strategies/metric-affine.mjs`; `src/hdc/strategies/metric-affine-elastic.mjs`; `src/hdc/strategies/exact.mjs`; `evals/runFastEval.mjs`.

## R04 • VSANash

Retrieval-aware sharding and placement. Twenty-three tests passed. Three HRR scenarios were rerun with eleven strategies each. The named AI review is a source for its explicitly attributed diagnostic suggestions, not for a complete development chronology.

Principal source paths: `src/sharding/equilibrium.mjs`; `src/eval/scenarios.mjs`; `tests/eval/runEval.mjs`; `antigravity_review.md`.

## R05 • VSABrains2

Structured episodic and reference-frame experiments. Thirty-three tests passed, and experiments 1, 2, 3, 4, and 6 were rerun. The first-trial consensus aggregation and exact-map query proxy are inspected implementation properties.

Principal source paths: `eval/exp1-alignment/run.mjs`;  
`eval/exp2-narrative/run.mjs`; `eval/exp3-rag/run.mjs`;  
`eval/exp4-consensus/run.mjs`; `eval/exp6-literature/run.mjs`.

## R06 • VSAText

Narrative schemas, text encoding, and structural evaluation. The selected runner passed 268 tests. The role-order collision and trait-count proxy were demonstrated by new probes. The Codex-attributed review documents its own runtime criticism.

Principal source paths: `src/vsa/encoder.mjs`; `src/evaluate/metrics.mjs`;  
`codex_review.md`; `README.md`.

## R07 • VSAVM

An executable VM and macro-oriented continuation experiment. The original selected suite passed 305 of 307 tests; a separate declaration-order

repair passed all 307. The standard evaluation reported eleven passing categories. These are different protocols.

Principal source paths: `src/training/outer-loop/macro-unit-model.mjs`;  
`evals/run.mjs`; `README.md`.

## R08 • SomaVM

Integer predictive learning and a supplied checkpoint. Seven tests passed. The checkpoint was evaluated with its own configuration on twenty included records, producing zero exact matches. Memory-enabled variants were not thereby evaluated.

Principal source paths: `README.md`; `src/cli/eval.mjs`; `Eval/data/core_preK_eval_large.jsonl`; `Eval/checkpoints/core_preK_long_max100.json`; `Eval/checkpoints/core_preK_long_max100.bin`.

## R09 • BSP

Explicit grouping and compression operations. Forty-six tests passed. Six abstract-primitives systems completed before the bounded sweep timed out. A new untrained-engine probe demonstrates the vacuous negation score. Compression-cost claims remain historical reports.

Principal source paths: `src/core/CompressionMachine.mjs`; `evals/abstract_primitives/06_negation/scorer.mjs`; `evals/abstract_primitives/06_negation/generator.mjs`; `experiments/COMPRESSION_INSIGHTS.md`; `experiments/SESSION_2026-01-16_FINAL_SUMMARY.md`.

## R10 • UBHNL

Typed parsing, compilation scaffolding, and canonical structures. The default development runner contains no implemented development tests. The separate typed-AST checks passed. Broader solver ambitions in the design documents remain distinct from executed capabilities.

Principal source paths: `src/index.mjs`; `src/kernel/store.mjs`; `src/dsl/ast.mjs`; `src/compiler/ubh-compiler.mjs`; `src/devtests/run.mjs`; `src/devtests/typed-ast-tests.mjs`.



## R11 • CNL

An executable constrained language, transactional compilation, provenance-related structures, and bounded planning. Eighty-five tests and eleven smoke suites passed. This does not establish unrestricted natural-language translation.

Principal source paths: `src/session/cnl-session.mjs`;  
`src/provenance/justifications.mjs`; `evals/runSmoke.mjs`; `README.md`.

## R12 • HoloProof

Formal proposal, world management, retrieval, and solver orchestration. Seventy-three tests passed. The live smoke matrix produced twelve dependency errors. The independent finite-Boolean probe tests the real orchestration but is not a third-party SMT solver replication.

Principal source paths: `src/sdk/reasoning/queryExecutor.mjs`; `src/sdk/reasoning/expansionPolicy.mjs`; `src/sdk/solver/smtEmitter.mjs`;  
`eval/runEval.mjs`; `README.md`.

## R13 • autoResearchLib

A bounded frontier of alternative theories and question policies. Eighteen of nineteen tests passed; the failure concerns expected sibling-project discovery. All seven experiment runners completed. Their regenerated tables and trace artifacts support the analysis of recovery and harmful contraction.

Principal source paths: `README.md`; `scripts/run-experiments.mjs`;  
`experiments/experiment1/run.mjs`; `experiments/experiment4/run.mjs`;  
`experiments/experiment5/run.mjs`; `experiments/experiment6/run.mjs`;  
`experiments/experiment7/run.mjs`.

## R14 • vsaAbduction / Eos Gate

The semantic invariant experiment was regenerated. Source and controls support a restricted additive-law claim, not open-ended scientific induction. The assistance disclosure supports only the authorship and

intervention statements it explicitly records. The dual-readout study is a new audit experiment.

Principal source paths: `src/world.js`; `src/abductor.js`;  
`scripts/run_experiment.js`; `test/verify_results.js`; `results/summary.json`;  
`docs/human_review_and_agent_use.md`.

## **R15 • vsaLLM / VSA-PathMoE**

Request-level sparse neural routing and a controlled deployment protocol. Twenty-eight Node tests and fifty-two Python tests passed. A complete frozen training-to-deployment result was not produced in this audit. The repository explicitly distinguishes protocol readiness from superiority evidence.

Principal source paths: `README.md`; `python/vsa_bench/router.py`;  
`python/vsa_bench/model.py`; `docs/specs/DS003-experimental-methodology.md`; `docs/specs/DS010-dgx-spark-handoff.md`.

## Appendix B. Audit artifacts and reproduction boundaries

### A01 • Input identity and selection

archive\_manifest.json records the fifteen original archives with full SHA-256 hashes, member counts, and uncompressed sizes. The per-repository inventory files identify the relevant source and documentation paths available for inspection. Archive dates were not used to invent a development chronology. Untouched extracted snapshots were kept apart from execution copies.

### A02 • Repository test executions

test\_runs.json records the selected commands, exit codes, execution times, and log hashes for the main test pass. Additional VSAText and vsaLLM Python runs are recorded in the experiment metadata. Logs preserve all failures. Counts refer to each runner's own test accounting; the book does not combine incompatible notions of tests, suites, fixtures, and assertions into a universal success statistic.

### A03 • Original experiment reruns

experiment\_runs.json, extra\_runs.json, and extra2\_runs.json record the selected experiment commands and their outcomes. The package retains regenerated Eos results, the three VSANash scenario files, autoResearchLib's seven experiment result sets, the HoloProof error matrix, the SomaVM checkpoint report and configuration, and other selected outputs. The VSARepresentations matrix and incomplete BSP sweep remain available as console logs. Runtimes are not controlled hardware benchmarks.

## A04 • Algebra and growth probes

`new_probes/algebra_probe.mjs` imports the supplied implementations and writes `algebra_results.json`. It tests positive confidence scaling, full sparse binding, metric isometry and distributivity, conditional similarity baselines, elastic payload growth, exact shared-factor semantics, exact term expansion, and atom-prefix stability. Counterexamples demonstrate failed contracts, not a universal impossibility result for the representation family.

## A05 • Eos dual-readout experiment

`new_probes/eos_dual_readout.cjs` writes `eos_dual_results.json`. Data seed 424242 is fixed; dimensions are 16, 32, 128, 512, and 8192; each uses five codebook seeds. Both thresholds use training data only. The tested full-rank, unquantised linear encodings permit a dual readout, with floating-point coefficients. This is a causal diagnostic of the decoder, not a demonstration that HDC outperforms direct twelve-feature arithmetic.

## A06 • Semantic and scope probes

`new_probes/semantic_scope.mjs` writes `semantic_scope_results.json`. It demonstrates the equal text encodings, trait-count metric behaviour, untrained-engine negation score, and four scope-sensitive HoloProof cases. The Boolean adapter enumerates four assignments and parses the actual emitted assertions. It is independent of the query executor but deliberately limited; no claim of full SMT coverage follows.

## A07 • VSAVM repair and rerun

`new_probes/vsavg_budget_order.patch` moves the `budgetMs` declaration before its use. The original 305/307 result remains unchanged in the baseline log. `logs/patched_full_VSAVM.log` records the separate complete 307/307 test rerun. The patch changes declaration order, not the model's learning semantics. It establishes removal of the observed runtime failure, not language generalisation.

## A08 • Assertion and provenance register

`claim_ledger.json`, `source_register.json`, and `AUTHORSHIP_AND_SCOPE.md` connect the book's main statements to source or execution records and mark proposals separately. The companion verification script checks selected stored result invariants. It is a consistency check over the evidence package, not a substitute for rerunning the repository experiments.

The package includes the original supplied ZIPs under `source_archives/`. A preparation script verifies their hashes and extracts them into a local snapshot directory. A separate probe runner creates fresh outputs without overwriting the archived audit results. Node.js is needed for the probes; the preparation and evidence checks use Python's standard library. Large neural training, live model calls, and external solver installations remain outside this reproduction path. Dependencies for wider repository suites are not silently bundled or assumed.

# References and novelty boundaries

These checked primary sources establish context, not an exhaustive novelty search. Failure to find an identical system would not establish priority.

L01. Kleyko, D., Rachkovskij, D. A., Osipov, E., and Rahimi, A. A Survey on Hyperdimensional Computing aka Vector Symbolic Architectures, Part I: Models and Data Transformations. *ACM Computing Surveys*, 55(6), 2022. DOI: 10.1145/3538531. Preprint: arXiv:2111.06077. Relevant to the diversity of VSA models and the need to specify the representation and its operations.

L02. Kleyko, D., Rachkovskij, D. A., Osipov, E., and Rahimi, A. A Survey on Hyperdimensional Computing aka Vector Symbolic Architectures, Part II: Applications, Cognitive Models, and Challenges. *ACM Computing Surveys*, 55(9), 2023. DOI: 10.1145/3558000. Preprint: arXiv:2112.15424. Relevant to the distinction between a representation family and a complete cognitive architecture.

L03. Clarkson, K. L., Ubaru, S., and Yang, E. Capacity Analysis of Vector Symbolic Architectures. 2023. arXiv:2301.10352. Relevant to task-specific representation capacity and the connections with sketching and Bloom filters. It is not a theorem about the capacity of every proposed VSA extension.

L04. Frady, E. P., Kent, S., Olshausen, B. A., and Sommer, F. T. Resonator Networks for Factoring Distributed Representations of Data Structures. 2020. arXiv:2007.03748. Relevant as prior work on an additional mechanism for recovering factors of distributed structures.

L05. Camposampiero, G., Hersche, M., Terzić, A., Wattenhofer, R., Sebastian, A., and Rahimi, A. Towards Learning Abductive Reasoning using VSA Distributed Representations. *NeSy 2024*. arXiv:2406.19121. Relevant to ARLC and the existing combination of programmed knowledge and learned abstract rules.

L06. Carzaniga, F., Hersche, M., Schindler, K., and Rahimi, A. Practical Lessons on Vector-Symbolic Architectures in Deep Learning-Inspired Environments. *NeSy 2025, Proceedings of Machine Learning Research*, 284, 218–236. Relevant to linear readout and practical representation choices. The Eos decoder correction is not claimed as an invention of linear readout.

L07. Tomkins-Flanagan, E., and Kelly, M. A. Hey Pentti, We Did It!: A Fully Vector-Symbolic Lisp. 2025 preprint, arXiv:2510.17889; the record also identifies a MathPsych/ICCM 2024 abstract. Relevant to general computation with explicitly included cleanup memory. Expressivity does not establish practical language-model efficiency.

L08. Kymn, C. J., Kleyko, D., Frady, E. P., Bybee, C., Kanerva, P., Sommer, F. T., and Olshausen, B. A. Computing with Residue Numbers in High-Dimensional Representation. 2023. arXiv:2311.04872. Relevant as prior work on arithmetic-oriented extensions to the VSA operation family.

L09. Roller, S., Sukhbaatar, S., Szlam, A., and Weston, J. Hash Layers For Large Sparse Models. 2021. arXiv:2106.04426. Relevant to non-learned sparse routing and the need for stronger novelty claims than inexpensive deterministic expert selection.

L10. Willsey, M., Nandi, C., Wang, Y. R., Flatt, O., Tatlock, Z., and Panchekha, P. egg: Fast and Extensible Equality Saturation. *POPL* 2021. DOI: 10.1145/3434304. Preprint: arXiv:2004.03082. Relevant to structured expression sharing, equivalence, and rewrite-based optimisation.

L11. Green, T. J., Karvounarakis, G., and Tannen, V. Provenance Semirings. *Proceedings of the Twenty-Sixth ACM SIGMOD-SIGACT-SIGART Symposium on Principles of Database Systems*, 31–40, 2007. DOI: 10.1145/1265530.1265535. Relevant to algebraic accounts of input dependence; provenance algebra itself is not a new SOP contribution.

L12. Lu, C. [Chris], Lu, C. [Cong], Lange, R. T., Foerster, J., Clune, J., and Ha, D. The AI Scientist: Towards Fully Automated Open-Ended Scientific Discovery. 2024. arXiv:2408.06292. Prior work on automated research, not validation of this audit.

## Open research note

This edition is intentionally public at <http://scriptahub.com/>. The most useful feedback is technical: reproducible counterexamples, stronger baselines, alternative operator families, evidence that a claimed limitation is only an implementation artifact, or demonstrations that one of the open experiments can be pushed substantially further. The book should be read as a frozen analysis of a research phase, not as the final statement of the programme.